KLAI

Machine Learning: instructor guide

Session-by-session plans, answers and checks for all 24 classes, the four labs and the optional extension.

Holiday Data and AI Camp | The National ICT Innovation Hub, Nakawa
Jotham Martin Wambi, main tutor

Contents

Private to the instructor. Each guide has the 180-minute plan, the answers, the code and the common errors for one session. Use the bookmarks panel to jump to a session.

F01	F01 Numbers, names and predicting Python output
F02	F02 Lists, vectors and NumPy arrays
F03	F03 Variation, proportions and probability
F04	F04 Matrices, weighted sums and slope
ML01	Your first instruction and your first prediction
ML02	Values, names and types
ML03	Arithmetic, converting text and reading errors
ML04	Decisions and a hand-written flower rule
ML05	Lists, dictionaries and loops
ML06	Functions and checks that fail loudly
ML07	Tables and reading a CSV file
ML08	Select, filter and calculate new columns
ML09	Mean, median, range and proportions
ML10	Cleaning without hiding evidence
ML11	Charts that show what the numbers say
ML12	Show your first chart and choose your project question
ML13	Features, targets and an honest baseline
ML14	Predicting a number with a line
ML15	A complete regression experiment
ML16	Predicting a category with a decision tree
ML17	Mistakes, overfitting and fair comparisons
ML18	A fair comparison: does your model beat the plain guess?
ML19	Project data and a safe plan
ML20	Build the project baseline
ML21	Compare, explain and critique
ML22	The final run: your project, ready to share
ML23	Explain the project to a parent
ML24	Parent showcase and next steps

F01: F01 Numbers, names and predicting Python output

Week 1, fourth teaching day. Complete ML03 first.

Today's question. Can I predict every line Python prints before I run it?

WHERE THIS IS USED

A good programmer reads code and predicts its output before pressing run. A mobile money app keeps an amount, a name and a date in named boxes, and each one must be right. Can you predict every line before it runs?

Words for today

Word	What it means	Think of it like this
scalar	One number.	One price on a label.
name	A label that points to a value.	A tag on a box.
top to bottom	Python reads one line after another and does not look ahead.	Reading a recipe in order.
update	Giving an existing name a new value.	Crossing out a price and writing a new one.
string	Text inside quotation marks.	A word on a label.
prediction on paper	Writing what you expect before you run.	Guessing the match score before kick-off.

Learn the idea

Python reads instructions from the top to the bottom, one at a time. A name points to a value. Changing a name does not change a result that was already calculated. Multiplication uses * and division uses /. The text in quotation marks is a string even when it looks like a number. print shows its input.

Start on paper

Draw three pens and label each UGX 1,000. Add them by hand: 1,000 + 1,000 + 1,000 is 3,000. Explain why 3 * 1000 gives the same total. Then write on paper every line the code below will print, before you run it.

Worked example

```
units = 3
price = 1000
total = units * price
print(total)
print(total / units)
print("1000")
units = 4
print(total)
total = units * price
print(total)
```

Run it, then compare: 3000, 1000.0, 1000, 3000 and 4000, on separate lines. The quoted 1000 is text. Changing units alone does not recalculate an earlier total.

Your turn

In a new code cell calculate the price of 5 books at UGX 3000 each and print it. Change the quantity to 2 and recalculate. In a text cell explain why the old total did not change until you recalculated it. Save F01.ipynb.

Run all cells from the top, save, close and reopen. Show the instructor your output and explanation.

Tutor pacing

10 minutes recall; 20 minutes paper example; 20 minutes run the code slowly together; 10 minutes break; 30 minutes learner variation; 20 minutes individual explanation and repair; 10 minutes save and exit check. Total 120. Visit all five learners and ask what each symbol and unit means before asking them to change it.

Reference: 15000, then 6000. A calculation runs when its line runs and it does not update by itself. Ask the learner to point to the changed input and rerun from the first cell.

Repair: If quotation marks surround a quantity, remove them so it is a number. If a total looks stale, rerun the line that calculates it. Do not move on until the learner can predict both totals.

Carry unfinished work into the next core session. Lab work goes in that week's build log. Give one strength and one next step.

Online version

Foundation labs run online as designed: two hours live on video with screen sharing, with the same minutes as the tutor pacing above and a short break away from the screen. There is no extra build task. Unfinished work carries into the next core class. Ask each learner to share their screen at least three times, to say their paper prediction aloud before they run the code, and to hold up their notebook or paper to the camera. See handbook/HYBRID_DELIVERY.md.

F02: F02 Lists, vectors and NumPy arrays

Week 2, fourth teaching day. Complete ML06 first.

Today's question. Why do the numbers in a list need a fixed order and a unit?

> **WHERE THIS IS USED**
> A row of daily temperatures or match scores only makes sense in a fixed order, with a unit. Weather stations and football analysts keep their numbers that way. What would break if the order of a list changed?

Words for today

Word	What it means	Think of it like this
scalar	A single number.	One mark.
vector	An ordered list of numbers whose positions have meanings.	A home described by area, then rooms.
NumPy array	A list of numbers that can be calculated with all at once.	A whole row of numbers worked on together.
shape	The size of an array, such as (2,) for two numbers.	Counting the boxes in a row.
element-wise	Doing the same calculation to every number.	Adding the same bonus to every pupil's mark.
unit	What a number measures, such as square metres.	Shillings, kilograms or metres.

Learn the idea

A scalar is one number. A vector is an ordered group of numbers, and the order gives each number its meaning: [40, 2] could be a home with an area of 40 square metres and 2 rooms. A plain Python list can hold a vector, but adding two lists joins them. NumPy arrays calculate element by element. np.array makes an array and its shape tells how many numbers it holds.

Start on paper

Draw two labelled boxes: area_m2 and rooms. Put 40 and 2 in them. Swap the numbers. Is it still the same home? No, because the order gives the numbers their meanings. Then write what you expect from [40, 2] + [10, 1] and from adding the two as arrays.

Worked example

```
import numpy as np
home = [40, 2]
print(len(home))
joined = home + [10, 1]
print(joined)
home_array = np.array([40, 2])
print(home_array.shape)
added = home_array + np.array([10, 1])
doubled = home_array * 2
print(added)
print(doubled)
```

Run it, then compare: 2, then [40, 2, 10, 1] (the lists are joined), then (2,), then [50 3] and [80 4] (calculated one by one).

Your turn

Make a NumPy array for a home of 65 square metres and 3 rooms. Print its shape. Add a text cell that labels the units of each position. Add the item counts [1, 4, 2] with a loop and then with .sum(). Save F02.ipynb.

Run all cells from the top, save, close and reopen. Show the instructor your output and explanation.

Tutor pacing

10 minutes recall; 20 minutes paper example; 20 minutes run the code slowly together; 10 minutes break; 30 minutes learner variation; 20 minutes individual explanation and repair; 10 minutes save and exit check. Total 120. Visit all five learners and ask what each symbol and unit means before asking them to change it.

Reference: Shape (2,); a total of 7 both ways. Do not add area to rooms: the units differ. Accept any loop variable name if the learner can explain it.

Repair: An IndexError means a position outside the array. [40, 2] has positions 0 and 1 only. If a learner expects list addition to add numbers, rerun the joined line and compare it with the array line.

Carry unfinished work into the next core session. Lab work goes in that week's build log. Give one strength and one next step.

Online version

Foundation labs run online as designed: two hours live on video with screen sharing, with the same minutes as the tutor pacing above and a short break away from the screen. There is no extra build task. Unfinished work carries into the next core class. Ask each learner to share their screen at least three times, to say their paper prediction aloud before they run the code, and to hold up their notebook or paper to the camera. See handbook/HYBRID_DELIVERY.md.

F03: F03 Variation, proportions and probability

Week 3, fourth teaching day. Complete ML09 first.

Today's question. Two groups can have the same mean. How can I tell them apart?

> **WHERE THIS IS USED**
> Forecasts speak in chances, and clinics report the share of patients with a problem. Two groups can have the same average and still look very different. How would you tell two football teams apart if both score the same average?

Words for today

Word	What it means	Think of it like this
mean	The total shared out equally.	Sharing a bag of mandazi.
deviation	A value minus the mean; it can be negative.	How far a mark is from the class average.
standard deviation	A typical size of the deviations, in the original unit.	How spread out the scores are.
proportion	A part divided by the whole.	3 blue counters out of 5.
probability	How likely an outcome is under stated conditions.	A fair die gives a 6 about once in six throws.
observed fraction	What actually happened in a limited number of tries.	Ten throws of a die.

Learn the idea

The mean is a total divided by a count. Two groups can have the same mean and different spread. A deviation is a value minus the mean. NumPy finds the mean and the standard deviation of an array: values.mean() and values.std(). Here the numbers describe the whole small group. A proportion is a part divided by a whole, and times 100 it becomes a percentage. A probability describes a chance under stated conditions. An observed fraction estimates it and does not promise the next outcome.

Start on paper

Put four cards on the table: 2, 2, 6, 6. The mean is 4. The deviations are -2, -2, 2, 2. The standard deviation is 2. Now four cards 4, 4, 4, 4: the same mean, no spread. For a fair six-sided die two of six faces are above 4, so the probability is 2/6. Ten real throws need not give exactly that fraction.

Worked example

```
import numpy as np
values = np.array([2, 2, 6, 6])
mean = values.mean()
deviations = values - mean
spread = values.std()
steady = np.array([4, 4, 4, 4])
print(mean, deviations, spread)
print(steady.mean(), steady.std())
print(2 / 6, 2 / 6 * 100)
```

Run it, then compare: Line one: mean 4.0, deviations minus 2, minus 2, 2 and 2, spread 2.0. Line two, the steady group: mean 4.0, spread 0.0. Line three: 0.3333 and 33.33, a chance of 1 in 3, or 33.3 percent.

Your turn

Repeat for [1, 1, 5, 5]. Explain the mean and spread in words. A bag has 3 blue and 2 red counters. State the chance of blue and its percentage, assuming each counter is equally likely. Save F03.ipynb.

Run all cells from the top, save, close and reopen. Show the instructor your output and explanation.

Tutor pacing

10 minutes recall; 20 minutes paper example; 20 minutes run the code slowly together; 10 minutes break; 30 minutes learner variation; 20 minutes individual explanation and repair; 10 minutes save and exit check. Total 120. Visit all five learners and ask what each symbol and unit means before asking them to change it.

Reference: Mean 3.0, standard deviation 2.0, probability of blue 3/5 = 0.6 = 60%. NumPy's std divides by the count, which suits a group described in full.

Repair: If the deviations cancel to zero, the learner added the signed deviations without squaring or using std. If a percentage is above 100, the whole was divided by the part.

Carry unfinished work into the next core session. Lab work goes in that week's build log. Give one strength and one next step.

Online version

Foundation labs run online as designed: two hours live on video with screen sharing, with the same minutes as the tutor pacing above and a short break away from the screen. There is no extra build task. Unfinished work carries into the next core class. Ask each learner to share their screen at least three times, to say their paper prediction aloud before they run the code, and to hold up their notebook or paper to the camera. See handbook/HYBRID_DELIVERY.md.

F04: F04 Matrices, weighted sums and slope

Week 4, fourth teaching day. Complete ML12 first.

Today's question. How does a model turn two measurements into one prediction?

> **WHERE THIS IS USED**
> A model turns measurements into one number by giving each a weight, like a rent that grows with size and rooms. A ride fare grows with distance and time. Which input would you give the biggest weight when pricing a ride?

Words for today

Word	What it means	Think of it like this
matrix	A table of numbers with rows and columns.	A page of a register.
shape	Rows by columns.	How many lines and headings.
weight	A number that says how much an input counts.	How many points each goal is worth.
weighted sum	Multiply each input by its weight and add the results.	Adding up the points from every goal.
intercept	The starting amount before any input is added.	A fixed fee.
slope	How much the answer changes when one input rises by one unit.	Extra rent for each extra square metre.

Learn the idea

A matrix is a rectangular table of numbers, and its shape is rows by columns. In NumPy a 2-D array holds one home on each row: [[40, 2], [60, 3]] has two homes with two measurements each. A weighted sum multiplies each input by its weight and adds the results. np.dot does this for every row at once. In the invented rule rent = 100000 + 3000 * area + 20000 * rooms, 100000 is the intercept and 3000 and 20000 are weights. The slope for area is the change in rent for one extra square metre, with rooms held fixed. These numbers are supplied for teaching. A model later learns weights from examples.

Start on paper

Draw a table with two rows and the columns area and rooms. For [40, 2] calculate 3000 * 40 + 20000 * 2 = 160000, then add 100000 to get 260000. Raise the area by 1 with rooms fixed: the answer rises by 3000. Label the columns before you code.

Worked example

```
import numpy as np
homes = np.array([[40, 2], [60, 3]])
weights = np.array([3000, 20000])
intercept = 100000
print(homes.shape)
print(homes[0, 1])
predictions = intercept + np.dot(homes, weights)
print(predictions)
```

Run it, then compare: Line one: (2, 2), the shape of two rows and two columns. Line two: 2, the rooms of home 0. Line three: 260000 and 340000, the rents predicted for the two homes.

Your turn

Add the home [50, 2] as a third row. Predict its result on paper before you run. State the new shape and what each weight means. Explain why these numbers say nothing about real rents in Kampala. Save F04.ipynb.

Run all cells from the top, save, close and reopen. Show the instructor your output and explanation.

Tutor pacing

10 minutes recall; 20 minutes paper example; 20 minutes run the code slowly together; 10 minutes break; 30 minutes learner variation; 20 minutes individual explanation and repair; 10 minutes save and exit check. Total 120. Visit all five learners and ask what each symbol and unit means before asking them to change it.

Reference: Shape (3, 2). The third prediction is 290000. 3000 is shillings per square metre and 20000 is shillings per room, each holding the other input fixed. The rule is invented.

Repair: If a learner writes homes * weights, show that it multiplies matching elements and does not add them, then use np.dot. If the column meaning is unclear, label the paper table before coding. Recheck independent predictions before ML13.

Carry unfinished work into the next core session. Lab work goes in that week's build log. Give one strength and one next step.

Online version

Foundation labs run online as designed: two hours live on video with screen sharing, with the same minutes as the tutor pacing above and a short break away from the screen. There is no extra build task. Unfinished work carries into the next core class. Ask each learner to share their screen at least three times, to say their paper prediction aloud before they run the code, and to hold up their notebook or paper to the camera. See handbook/HYBRID_DELIVERY.md.

ML01: Your first instruction and your first prediction

Private. Keep this file out of the student folder.

Objective. By the end: you can run, save and reopen a notebook, and you can say what a prediction is using the rent example.

Question of the day. Can a computer guess a rent after seeing 120 homes?

> **WHERE THIS IS USED**
> Property apps and agents guess a fair rent from the size and area of a home. A model does the same after studying homes that were already rented, and today you build the smallest version. What rent would you guess for a home you know?

Prerequisite: START_HERE.md.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Where it overruns: Laptop and environment problems can use the first 20 minutes. Keep block B short and finish with the rent estimate for 80 square metres.

Block A. Run your first instruction

Open ML01.ipynb in Jupyter. Click the first code cell and press Shift+Enter.

Ask for the prediction first: Write on paper what you think these two lines will show.

```
print("Hello, data")
print(3 + 2)
```

Correct result: The first line shows Hello, data and the second shows 5. A notebook is a page of text cells and code cells. Shift+Enter runs a code cell and its output appears underneath. print is a ready-made instruction, and the brackets hold what it shows.

Investigate, with answers:

- What happens if you click a text cell and press Shift+Enter? Answer: Nothing runs. A text cell only explains; only a code cell runs Python.

- Why does Hello, data sit inside quotation marks while 3 + 2 does not? Answer: Quotation marks tell Python the characters are text. Without them Python calculates.

Modify: Change Hello, data to your own nickname and run the cell again. Answer: The first line now shows your nickname.

Block B. Run a ready-made prediction

This model has looked at 120 invented homes and learned how rent follows floor area. You do not need to understand it yet. Session 13 opens it up.

Ask for the prediction first: Before you run it, write the monthly rent in UGX you expect for a home of 50 square metres.

```
import pandas as pd
from sklearn.linear_model import LinearRegression
homes = pd.read_csv("synthetic_homes.csv")
model = LinearRegression().fit(homes[["area_m2"]], homes["rent_ugx"])
pred = model.predict(pd.DataFrame({"area_m2": [50]}))[0]
print("Estimated rent for 50 square metres:", round(pred))
```

Correct result: The model estimates about UGX 309,260 a month. It drew a line through the 120 homes and read the rent at 50 square metres. Guessing a number from examples is called regression, and you will build one yourself in session 14. The homes are invented, so the estimate says nothing about real rents.

Investigate, with answers:

- The home grows from 50 to 100 square metres. What do you expect the estimate to do? Answer: It rises. A 50 square metre increase adds about UGX 150,000 here.
- Is the estimate certain? Answer: No. It is a guess from 120 invented homes and can be wrong for any real home.
- Does this model know anything about Kampala? Answer: No. It only knows the 120 invented rows in the file.

Modify: Change 50 to 100 and run the cell again. Answer: The estimate becomes about UGX 459,084.

```
pred100 = model.predict(pd.DataFrame({"area_m2": [100]}))[0]
```

Make: reference

```
import pandas as pd
from sklearn.linear_model import LinearRegression
print("Amina")
print(7 + 4)
homes = pd.read_csv("synthetic_homes.csv")
model = LinearRegression().fit(homes[["area_m2"]], homes["rent_ugx"])
pred80 = model.predict(pd.DataFrame({"area_m2": [80]}))[0]
print(round(pred80))
```

Build log entry to look for: ML01.ipynb with a nickname, 11, and a rent estimate for 80 square metres, reopened and saved.

Misconceptions to listen for

- A model that gives a number must be correct.
- An error message means I broke the computer.
- The notebook runs by itself when I type.

Questions to ask, with expected answers

- What did the model need before it could guess a rent? Expected: Examples: the 120 homes with their areas and rents.
- Why should we distrust the estimate? Expected: The homes are invented, and nobody has tested the model yet.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Welcome, camera and sound check, and a notebook that opens
35	Block A, Run your first instruction: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, Run a ready-made prediction: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Run your first instruction): Change Hello, data to your own nickname and run the cell again.
2. Modify, Block B (Run a ready-made prediction): Change 50 to 100 and run the cell again.
3. Make: In a new code cell print your nickname and the result of 7 + 4. In another cell estimate the rent for 80 square metres. Save as ML01.ipynb, close it and reopen it. On your progress map, write one topic you love, such as football, music or farming. It will grow into your project. With the instructor, read AI_USE_POLICY.md: in weeks 1 to 6 all core work is your own, with no AI-generated solutions.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What did the model need before it could guess a rent? Expected: Examples: the 120 homes with their areas and rents.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML02: Values, names and types

Private. Keep this file out of the student folder.

Objective. By the end: you can create whole numbers, decimals and text, and predict what happens when a name is given a new value.

Question of the day. What kind of thing is each part of a sale?

> **WHERE THIS IS USED**
> A mobile money app keeps the amount as a number and the phone number as text. Mixing them up would cause real mistakes. What would go wrong if an app added two phone numbers together?

Prerequisite: ML01 Your first instruction and your first prediction.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Block A. Names point to values

A shop sells 3 items at UGX 2000 each. Run the cell and read each line of output.

Ask for the prediction first: Predict the three lines the cell will print.

```
price = 2000
quantity = 3
cost = price * quantity
print(cost)
print(type(price))
print(type("2000"))
```

Correct result: The cell prints 6000, then <class 'int'>, then <class 'str'>. price and quantity are variables. 2000 is an integer. The quoted 2000 is a string, which is text. The equals sign stores the value on its right under the name on its left.

Investigate, with answers:

- Why is type("2000") a string? Answer: Quotation marks make it text. Python cannot multiply it like an amount of money.
- What does the equals sign do in cost = price * quantity? Answer: It calculates the right side first, then stores the answer under the name cost.

Modify: Change quantity to 5 and run the whole cell again. Answer: cost becomes 10000.

```
quantity = 5
cost = price * quantity
```

Block B. A name keeps its value until you change it

Run the cell and watch what the printed total does when quantity changes.

Ask for the prediction first: Predict the three numbers printed. Will the second one change after quantity changes?

```
price = 2500.5
quantity = 2
total = price * quantity
print(total)
quantity = 4
print(total)
total = price * quantity
print(total)
```

Correct result: The cell prints 5001.0, then 5001.0, then 10002.0. 2500.5 is a float. Changing quantity does not change total until the line that calculates total runs again. Python works from the top of the cell to the bottom.

Investigate, with answers:

- Why does the second print still show 5001.0? Answer: total was calculated when quantity was 2. Python does not recalculate by itself.
- What line makes the third number different? Answer: The second total = price * quantity line, which runs with quantity equal to 4.

Modify: Set quantity to 10, recalculate total and print it. Answer: The total is 25005.0.

```
quantity = 10
total = price * quantity
```

Make: reference

```
price = 2500.5
quantity = 2
print(price * quantity)
print(type("2500.5"))
# 5001.0. With quotation marks 2500.5 is a string, which is text and not an amount.
```

Build log entry to look for: ML02.ipynb with the total 5001.0 and a sentence explaining the quotation marks.

Misconceptions to listen for

- = asks whether two things are equal.
- Quotation marks around a number change nothing.

- Changing quantity changes total automatically.

Questions to ask, with expected answers

- What does the equals sign do? Expected: It stores the value on the right under the name on the left.
- Why is 5001.0 shown with a decimal point? Expected: Multiplying by a float gives a float.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Names point to values: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, A name keeps its value until you change it: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Names point to values): Change quantity to 5 and run the whole cell again.
2. Modify, Block B (A name keeps its value until you change it): Set quantity to 10, recalculate total and print it.
3. Make: Create a decimal price of 2500.5 and a quantity of 2. Print the total. Write one sentence on why "2500.5" with quotation marks is different.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What does the equals sign do? Expected: It stores the value on the right under the name on the left.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML03: Arithmetic, converting text and reading errors

Private. Keep this file out of the student folder.

Objective. By the end: you can calculate with the five basic operators, turn text into a number, and find the line an error points to.

Question of the day. How do I turn the text of a price into a number I can calculate with?

> **WHERE THIS IS USED**
> Anything you type into a form arrives as text. The app has to turn it into a number before it can add, compare or charge. Where in the apps you use does text quietly become a number?

Prerequisite: ML02 Values, names and types.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Where it overruns: Students spend longest on reading errors. Cut the extra operator practice before cutting the error routine.

Block A. The basic operators

Run the cell. Notice the difference the brackets make.

Ask for the prediction first: Predict the four numbers before you run.

```
price = 3000
total = price * 4
share = price / 4
no_brackets = price - 500 * 2
with_brackets = (price - 500) * 2
print(total, share, no_brackets, with_brackets)
```

Correct result: The cell prints 12000, 750.0, 2000 and 5000. + adds, - subtracts, * multiplies and / divides. Multiplication happens before subtraction, so the brackets change the answer from 2000 to 5000. Brackets are called parentheses.

Investigate, with answers:

- Why do 2000 and 5000 differ? Answer: Without brackets 500 * 2 happens first. With brackets 3000 - 500 happens first.
- Why is share 750.0 and not 750? Answer: Division always gives a float in Python.

Modify: Write the calculation for 3 books at UGX 3000 plus 2 pens at UGX 1000 in one line. Answer: 3 * 3000 + 2 * 1000 gives 11000.

```
basket = 3 * 3000 + 2 * 1000
```

Block B. Text becomes a number

A price arrives as text from a form. Run the cell and read the result.

Ask for the prediction first: Predict the output. Can Python multiply text by 4 and give 50.0?

```
# The price came in as text
text_price = "12.5"
price = float(text_price)
total = price * 4
print(total)
```

Correct result: The cell prints 50.0. float turns the text 12.5 into a number. The line starting with # is a comment, which Python ignores. If the text were abc, float would stop with a ValueError and the last line of the error would name the problem.

Investigate, with answers:

- Which line of an error message do you read first? Answer: The last line. It names the kind of error and what went wrong.
- What does float("12,5") do? Answer: It stops with a ValueError, because Python does not accept a comma inside a number.

Modify: Change the text price to "20" and run the cell again. Answer: The total becomes 80.0.

```
text_price = "20"
price = float(text_price)
total = price * 4
```

Make: reference

```
cost_of_four = float("12.5") * 4
print(cost_of_four)
# 50.0. Dividing by zero asks how many times zero fits into a number, and there is no
    answer.
```

Build log entry to look for: ML03.ipynb with the prediction written first, the result 50.0, and a sentence on division by zero.

Misconceptions to listen for

- Brackets are decoration.
- An error is a judgement about me.

- Text that looks like a number can be multiplied.

Questions to ask, with expected answers

- What is the difference between 12.5 and "12.5"? Expected: The first is a number. The second is text.
- Where do you start reading an error? Expected: At the last line, then the line number it names.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, The basic operators: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, Text becomes a number: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (The basic operators): Write the calculation for 3 books at UGX 3000 plus 2 pens at UGX 1000 in one line.
2. Modify, Block B (Text becomes a number): Change the text price to "20" and run the cell again.
3. Make: Convert "12.5" to a number and calculate the price of four items. Predict the answer before you run it. Explain why nobody can divide by zero.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What is the difference between 12.5 and "12.5"? Expected: The first is a number. The second is text.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML04: Decisions and a hand-written flower rule

Private. Keep this file out of the student folder.

Objective. By the end: you can write an if, elif and else decision, test it at its boundary, and count how often a simple rule is right.

Question of the day. How often does a three-line rule name the right flower?

> **WHERE THIS IS USED**
> Apps decide with rules such as: if the balance is too low, refuse the payment. Botanists also tell flowers apart by measuring petals, and today you write a three-line rule that does it. Can your rule beat a guess?

Prerequisite: ML03 Arithmetic, converting text and reading errors.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Where it overruns: The flower table can overrun. Fill in only the first four flowers together and let students finish the rest in the Make task.

Block A. A decision with if and else

A student has UGX 10000 and an item costs UGX 8000. Run the cell, then change the cost.

Ask for the prediction first: Predict the message that will print.

```
budget = 10000
cost = 8000
if cost <= budget:
    message = "Within budget"
else:
    message = "Over budget"
print(message)
```

Correct result: The cell prints Within budget. cost <= budget is a comparison and its answer is True, which is a Boolean. if runs the indented lines when the answer is True and else runs them when it is False. The four spaces at the start of the lines are indentation.

Investigate, with answers:

- What is the answer to 5000 >= 5000? Answer: True. >= means at least.

- What does = do compared with ==? Answer: = stores a value under a name. == asks whether two values are equal.

Modify: Set cost to 12000 and run the decision again. Answer: The message is Over budget.

```
cost = 12000
if cost <= budget:
    message = "Within budget"
else:
    message = "Over budget"
```

Block B. A rule that names a flower

The iris data records flowers of three species. Petal length helps tell them apart. Run the rule for one flower, then try the others on the table.

Ask for the prediction first: Look at the rule. Predict which species it names for a petal length of 4.7 centimetres.

```
petal_length = 4.7
if petal_length < 2.5:
    guess = "setosa"
elif petal_length < 4.8:
    guess = "versicolor"
else:
    guess = "virginica"
print(guess)
```

Correct result: The cell prints versicolor. elif asks a second question only when the first answer was False. Try the eight petal lengths. The true species are setosa twice, versicolor three times and virginica three times. The rule gets seven right and misses the 4.9 flower.

Investigate, with answers:

- Which of the eight flowers does the rule get wrong? Answer: The one with petal length 4.9. It is versicolor, and the rule says virginica.
- How would you move the rule so that 4.9 is right? Answer: Raise the 4.8 limit to 5.0. Then check whether the 5.1 flower is still right.
- What is the accuracy on these eight flowers? Answer: 7 right out of 8, which is 87.5%.

Modify: Set petal_length to 4.9 and run the rule. Answer: It prints virginica, which is wrong for this flower.

```
petal_length = 4.9
if petal_length < 2.5:
    guess = "setosa"
elif petal_length < 4.8:
    guess = "versicolor"
else:
    guess = "virginica"
```

Make: reference

```
budget = 5000
if budget >= 5000:
    decision = "Buy"
else:
    decision = "Save"
print(decision)
# 4999 gives Save; 5000 and 5001 give Buy. The flower rule is right for 7 of 8 flowers.
```

Build log entry to look for: ML04.ipynb with the ticket decision tested at three budgets and the flower count 7 out of 8.

Misconceptions to listen for

- If a rule is right on a few examples it is right everywhere.
- A single = tests equality.
- The lines after if run even when the answer is False.

Questions to ask, with expected answers

- Why test 4999, 5000 and 5001? Expected: The change happens at the boundary, and boundaries are where rules break.
- What does the flower rule teach us about prediction? Expected: Even a simple rule is wrong sometimes, so every rule needs counting on examples.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, A decision with if and else: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, A rule that names a flower: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (A decision with if and else): Set cost to 12000 and run the decision again.
2. Modify, Block B (A rule that names a flower): Set petal_length to 4.9 and run the rule.
3. Make: A ticket costs UGX 5000. Write a decision that prints Buy when the budget is at least 5000 and Save otherwise. Test 4999, 5000 and 5001. Then say how many of the eight flowers your flower

rule names correctly.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: Why test 4999, 5000 and 5001? Expected: The change happens at the boundary, and boundaries are where rules break.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML05: Lists, dictionaries and loops

Private. Keep this file out of the student folder.

Objective. By the end: you can read a list position and a dictionary key, and trace a loop that adds up values.

Question of the day. How do I repeat one calculation for every item in a list?

> **WHERE THIS IS USED**
> A football league table is a list of teams with points, and a program loops over it to add up every match. Your mobile money statement works the same way. What would you loop over in your favourite sport?

Prerequisite: ML04 Decisions and a hand-written flower rule.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Block A. Lists and dictionaries

A tuck shop keeps three prices in a list and one product in a dictionary.

Ask for the prediction first: Predict what prints. Which number is prices[0]?

```
prices = [1000, 2000, 3000]
product = {"name": "Juice", "price": 2000}
first = prices[0]
juice_price = product["price"]
print(first)
print(juice_price)
print(len(prices))
```

Correct result: The cell prints 1000, 2000 and 3. Square brackets after a list give the item at that index, and the first index is 0. Square brackets after a dictionary give the value for that key. len counts the items in a list.

Investigate, with answers:

- Why is prices[0] the first price? Answer: Python counts positions from 0.
- What happens if you ask for prices[3]? Answer: An IndexError. The list has positions 0, 1 and 2 only.

Modify: Print the last price using its index. Answer: prices[2] gives 3000.

```
last = prices[2]
```

Block B. A loop that adds

Run the loop and follow how total grows after each pass.

Ask for the prediction first: Predict the four numbers printed.

```
prices = [1000, 2000, 3000]
total = 0
for price in prices:
    print(price * 2)
    total = total + price
print(total)
```

Correct result: The cell prints 2000, 4000, 6000 and then 6000. for price in prices takes one item at a time and runs the indented lines for it. total starts at 0 and grows by the item each pass: 1000, then 3000, then 6000.

Investigate, with answers:

- What is total after the second pass? Answer: 3000, which is 1000 plus 2000.
- What goes wrong if total = 0 is moved inside the loop? Answer: total restarts at 0 on every pass, so only the last item remains.

Modify: Change the list to [1500, 2500] and run the loop again. Answer: The final total is 4000.

```
prices = [1500, 2500]
total = 0
for price in prices:
    total = total + price
```

Make: reference

```
values = [2, 4, 6]
for value in values:
    print(value + 1)
item = {"name": "Book", "stock": 8}
print(item["stock"])
lengths = [1.4, 1.4, 4.7, 4.5, 4.9, 6.0, 5.1, 5.9]
short = 0
for length in lengths:
    if length < 2.5:
        short = short + 1
print(short)
# 3, 5, 7, then 8, then 2 short petals.
```

Build log entry to look for: ML05.ipynb with the three loop outputs, the stock 8, and the count 2.

Misconceptions to listen for

- The first item is at position 1.
- A dictionary keeps its values in a position order.

- A loop changes the original list.

Questions to ask, with expected answers

- What does position 0 mean? Expected: The first item in the list.
- Where should total = 0 go? Expected: Before the loop, so it starts once.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Lists and dictionaries: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, A loop that adds: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Lists and dictionaries): Print the last price using its index.
2. Modify, Block B (A loop that adds): Change the list to [1500, 2500] and run the loop again.
3. Make: Make the list [2, 4, 6] and print each value plus 1 with a loop. Make a dictionary with the keys name and stock and print the stock. Then loop over the eight petal lengths 1.4, 1.4, 4.7, 4.5, 4.9, 6.0, 5.1, 5.9 and count how many are below 2.5.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What does position 0 mean? Expected: The first item in the list.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML06: Functions and checks that fail loudly

Private. Keep this file out of the student folder.

Objective. By the end: you can write a function that returns a result, and check it against known answers with assert.

Question of the day. How do I turn my flower rule into a reusable tool and test it?

> **WHERE THIS IS USED**
> A fee calculator is a function: write it once and use it for every payment. Software teams also write checks with known answers, so a mistake shows up before a customer finds it. What check would you write for a fee calculator?

Prerequisite: ML05 Lists, dictionaries and loops.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Block A. A function and an assert

A function names a calculation so you can use it again. Run the cell. A passing assert prints nothing.

Ask for the prediction first: Predict what total(2000, 3) gives and what happens when the assert passes.

```
def total(price, quantity):
    return price * quantity

result = total(2000, 3)
print(result)
assert total(2000, 3) == 6000
```

Correct result: The cell prints 6000. def starts the function. price and quantity are parameters, and the 2000 and 3 you give it are arguments. return sends the answer back. assert stops with an error when the statement is false, and a passing assert prints nothing.

Investigate, with answers:

- What is the difference between return and print? Answer: return gives the value back to the program. print only shows it on the screen.
- What happens if you define total but never call it? Answer: Nothing runs. Defining a function does not run its lines.

Modify: Add an assert that total(1000, 0) equals 0. Answer: It passes silently, because 1000 times 0 is 0.

```
assert total(1000, 0) == 0
```

Block B. The flower rule as a function

Your rule from session 4 becomes a function. Three asserts test it, and a loop counts it on eight flowers.

Ask for the prediction first: Predict how many of the eight flowers the function names correctly.

```
def guess_species(petal_length):
    if petal_length < 2.5:
        return "setosa"
    elif petal_length < 4.8:
        return "versicolor"
    else:
        return "virginica"

assert guess_species(1.4) == "setosa"
assert guess_species(4.7) == "versicolor"
assert guess_species(6.0) == "virginica"
flowers = [[1.4, "setosa"], [1.4, "setosa"], [4.7, "versicolor"], [4.5, "versicolor"],
    [4.9, "versicolor"], [6.0, "virginica"], [5.1, "virginica"], [5.9, "virginica"]]
right = 0
for flower in flowers:
    if guess_species(flower[0]) == flower[1]:
        right = right + 1
print(right)
```

Correct result: The cell prints 7. The three asserts pass without a sound. Each flower in the list holds a petal length at position 0 and a species at position 1. The function is right for seven of eight flowers, the same count you found by hand.

Investigate, with answers:

- What does flower[0] give? Answer: The petal length, because position 0 is the first item.
- Why test the function on three known flowers before counting? Answer: If an assert fails you know the rule is broken before you trust any count.

Modify: Add the flower [4.9, "versicolor"] again to the list and recount. Answer: The count becomes 7 out of 9, because the 4.9 flower is wrong each time.

```
flowers = flowers + [[4.9, "versicolor"]]
right = 0
for flower in flowers:
    if guess_species(flower[0]) == flower[1]:
        right = right + 1
```

Make: reference

```
def double(number):
    return number * 2

assert double(4) == 8
assert double(0) == 0
assert double(-2) == -4
```

Build log entry to look for: ML06.ipynb with double defined, three passing asserts, and a clean run from the top.

Misconceptions to listen for

- Defining a function runs it.
- print and return do the same job.
- A passing assert should show a message.

Questions to ask, with expected answers

- What makes a good test case? Expected: A case where you already know the right answer, including a boundary such as 0.
- What does assert print when it passes? Expected: Nothing.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, A function and an assert: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, The flower rule as a function: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (A function and an assert): Add an assert that total(1000, 0) equals 0.
2. Modify, Block B (The flower rule as a function): Add the flower [4.9, "versicolor"] again to the list and recount.
3. Make: Define double(number) so it returns twice its input. Check double(4), double(0) and double(-2) with assert. Save the notebook and run all cells from the top.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What makes a good test case? Expected: A case where you already know the right answer, including a boundary such as 0.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML07: Tables and reading a CSV file

Private. Keep this file out of the student folder.

Objective. By the end: you can read a CSV file into a pandas table and say how many rows and columns it has.

Question of the day. How many sales are in the practice shop file, and what does each one record?

> **WHERE THIS IS USED**
> A shop till, a clinic register and a school timetable are all tables with rows and columns. Programs read such tables from files before they can learn anything. Which table in your life would you like to open in Python?

Prerequisite: ML06 Functions and checks that fail loudly.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Where it overruns: This session is the first with a library. Spend the first block on the tiny table and keep the file reading to the second block.

Block A. Build a small table

pandas is a library for tables. Run the cell and look at the table it prints.

Ask for the prediction first: Predict how many rows and columns the table has.

```
import pandas as pd
sales = pd.DataFrame({"item": ["Book", "Pen", "Book"], "units": [2, 5, 3]})
print(sales)
print(sales.shape)
```

Correct result: The table has 3 rows and 2 columns, so shape prints (3, 2). The headers are item and units. import pandas as pd loads the library and gives it the short name pd. A table made this way is a DataFrame. The dot after sales reaches a property that belongs to that table.

Investigate, with answers:

- What does the first number of shape count? Answer: Rows. The second number counts columns.
- Where do the headers come from? Answer: From the names in the curly brackets: item and units.

Modify: Add a fourth sale, Pen with 4 units, to both lists and run again. Answer: shape becomes (4, 2).

```
sales = pd.DataFrame({"item": ["Book", "Pen", "Book", "Pen"], "units": [2, 5, 3, 4]})
```

Block B. Read the practice shop file

camp_shop.csv sits beside your notebook. Read it into a table and look at the first rows.

Ask for the prediction first: Predict how many rows camp_shop.csv has and which headers it shows.

```
import pandas as pd
shop = pd.read_csv("camp_shop.csv")
print(shop.head())
print(shop.shape)
print(list(shop.columns))
```

Correct result: The table has 60 rows and 5 columns, so shape prints (60, 5). The headers are sale_id, date, item, units and unit_price_ugx. read_csv reads the file into a DataFrame and head shows its first five rows. The unit of money is in the header: unit_price_ugx is in shillings.

Investigate, with answers:

- Why is the unit written in the header name unit_price_ugx? Answer: A number without a unit is easy to misread. The header says the price is in shillings.
- What do the 60 rows stand for? Answer: Sixty sales. Each row is one sale.

Modify: Show the first 10 rows by writing 10 inside the brackets of head. Answer: Ten rows print.

```
first_ten = shop.head(10)
```

Make: reference

```
import pandas as pd
homes = pd.read_csv("synthetic_homes.csv")
print(homes.head())
print(homes.shape)
# 120 rows and 4 columns. Each row is one invented home. rent_ugx is monthly rent in
    shillings.
```

Build log entry to look for: ML07.ipynb with the first rows, the shape (120, 4), and two sentences on a row and the unit.

Misconceptions to listen for

- A DataFrame is only a picture that I cannot calculate with.
- The shape pair is the same as the first two rows.
- Every column has the same type.

Questions to ask, with expected answers

- What does shape return for a table of 20 sales and 5 headers? Expected: (20, 5).
- Why keep the CSV beside the notebook? Expected: read_csv looks for the file starting from the notebook folder.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Build a small table: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, Read the practice shop file: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Build a small table): Add a fourth sale, Pen with 4 units, to both lists and run again.
2. Modify, Block B (Read the practice shop file): Show the first 10 rows by writing 10 inside the brackets of head.
3. Make: Read synthetic_homes.csv. Show its first rows and its shape. Say what each row stands for and what unit rent_ugx is in.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What does shape return for a table of 20 sales and 5 headers? Expected: (20, 5).

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML08: Select, filter and calculate new columns

Private. Keep this file out of the student folder.

Objective. By the end: you can pick a column, keep rows that meet a condition, add a revenue column and total it by item.

Question of the day. Which item earned the most money in the practice shop?

> **WHERE THIS IS USED**
> A shopkeeper asks which item earned the most this week. Answering needs a filter and a new column: units times price. What would you ask of the sales of your own shop?

Prerequisite: ML07 Tables and reading a CSV file.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Block A. Select and filter

Read the practice shop and keep the large sales.

Ask for the prediction first: Predict how many of the 60 sales have 5 units or more.

```
import pandas as pd
shop = pd.read_csv("camp_shop.csv")
units = shop["units"]
big = shop[shop["units"] >= 5]
print(len(big))
print(units.sum())
```

Correct result: The cell prints 30 and 277. shop["units"] selects one column. The comparison inside the outer brackets gives True or False for every row, and only the True rows stay. That filter keeps 30 sales. All 60 sales together sold 277 units.

Investigate, with answers:

- Is the filter deleting the other 30 rows? Answer: No. The big table is a new, smaller table. shop still has all 60 rows.
- Why can the sum of units not tell us the money earned? Answer: The three items have different prices, so units and money are different quantities.

Modify: Filter the Juice sales with 5 units or more and count them. Answer: Seven sales.

```
juice_big = shop[(shop["item"] == "Juice") & (shop["units"] >= 5)]
n_juice_big = len(juice_big)
```

Block B. A revenue column and totals by item

Multiply units by price on every row, then add the money for each item.

Ask for the prediction first: Predict which item earns the most money. Is it the one with the most units?

```
shop["revenue_ugx"] = shop["units"] * shop["unit_price_ugx"]
total_revenue = shop["revenue_ugx"].sum()
by_item = shop.groupby("item")["revenue_ugx"].sum()
print(total_revenue)
print(by_item)
```

Correct result: The total is 550000 shillings. Book earns 297000, Juice 150000 and Pen 103000. Pen sold the most units, 103, but Book earns the most money because each Book costs 3000. Assigning to a new column name creates the column, and the groupby line adds the revenue of each item separately.

Investigate, with answers:

- Pen sold the most units. Why is it last for revenue? Answer: A pen costs only UGX 1000, so its units bring in little money.
- Do the three item totals add up to the overall total? Answer: Yes: 297000 + 150000 + 103000 is 550000. If they did not, something would be wrong.

Modify: Group the units column by item instead and print the totals. Answer: Book 99, Juice 75, Pen 103.

```
units_by_item = shop.groupby("item")["units"].sum()
```

Make: reference

```
import pandas as pd
shop = pd.read_csv("camp_shop.csv")
over3 = shop[shop["units"] >= 3]
total_units = over3["units"].sum()
mean_units = over3["units"].mean()
print(len(over3), total_units, mean_units)
# 42 rows, 247 units in all, about 5.88 units per sale. The total is for all the rows
    together and the mean is for one row.
```

Build log entry to look for: ML08.ipynb with 42 rows, total 247, mean about 5.88, and a sentence on the difference.

Misconceptions to listen for

- Filtering removes rows from the file.
- The item with the most units earns the most money.
- A mean is the same as a total.

Questions to ask, with expected answers

- Why do we check that the grouped totals add to the overall total? Expected: It catches rows lost or counted twice.
- Can this table tell us profit? Expected: No. It has no costs.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Select and filter: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, A revenue column and totals by item: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Select and filter): Filter the Juice sales with 5 units or more and count them.
2. Modify, Block B (A revenue column and totals by item): Group the units column by item instead and print the totals.
3. Make: Keep the rows with units of at least 3. Calculate the total units and the mean units per row. Say why these two numbers are different quantities.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: Why do we check that the grouped totals add to the overall total? Expected: It catches rows lost or counted twice.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML09: Mean, median, range and proportions

Private. Keep this file out of the student folder.

Objective. By the end: you can calculate the mean, median, range and a proportion, and say which one an extreme value moves most.

Question of the day. What is a typical number of units in a sale, and how spread out are the sales?

> **WHERE THIS IS USED**
> News reports quote the average income or the share of people who did something. One very large value can drag the mean up, while the median stays calm. Which would you quote for what most boda-boda rides on your road cost?

Prerequisite: ML08 Select, filter and calculate new columns.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Block A. Typical value and spread

Look at how many units each sale has. Run the cell and read the four numbers.

Ask for the prediction first: Predict whether the mean or the median of the units column is larger.

```
import numpy as np
import pandas as pd
shop = pd.read_csv("camp_shop.csv")
units = shop["units"]
mean_units = units.mean()
median_units = np.median(units)
spread = units.max() - units.min()
p75 = np.percentile(units, 75)
print(mean_units, median_units, spread, p75)
```

Correct result: The cell prints about 4.62, 4.5, 7 and 7.0. The mean is the total of 277 divided by 60 sales. The median 4.5 sits between the 30th and 31st values in order. The range is 8 minus 1. The 75th percentile 7 means three quarters of the sales have 7 units or fewer. NumPy is the library behind pandas for numbers, and np is its short name.

Investigate, with answers:

- Why is the median 4.5 and not a whole number? Answer: With 60 values the middle falls between two of them, so it is their average.
- What does the 75th percentile tell a shopkeeper? Answer: Three quarters of the sales had 7 units or fewer. Only 10 of the 60 sales had 8, the most any sale had, so stock for 7 or 8 units at a time.

Modify: Find the 25th percentile of units. Answer: 2.0.

```
p25 = np.percentile(units, 25)
```

Block B. One extreme value and a proportion

Add one very large value to a small list and see what moves. Then count a proportion.

Ask for the prediction first: Predict which moves more when 100 joins the list: the mean or the median.

```
small = pd.Series([10, 12, 14, 16])
bigger = pd.Series([10, 12, 14, 16, 100])
print(small.mean(), small.median(), bigger.mean(), bigger.median())
share_big = (units >= 5).sum() / len(units)
print(share_big, share_big * 100)
```

Correct result: The cell prints 13.0, 13.0, 30.4 and 14.0, then 0.5 and 50.0. The outlier 100 pulls the mean from 13 to 30.4 and moves the median only to 14. Half of all sales have 5 units or more: 30 out of 60 is the proportion 0.5, which is 50%.

Investigate, with answers:

- Which summary should you quote when one huge sale sits in the data? Answer: The median, because it hardly moves. Also report the outlier.
- Why must you name the whole when you give a proportion? Answer: 30 out of 60 and 30 out of 300 are very different. The denominator tells what the share is a share of.

Modify: Replace 100 with 1000 in bigger and print the mean and the median. Answer: The mean becomes 210.4 and the median stays 14.0.

```
bigger = pd.Series([10, 12, 14, 16, 1000])
```

Make: reference

```
rate_a = 12 / 20 * 100
rate_b = 15 / 50 * 100
print(rate_a, rate_b)
# 60.0 and 30.0. 15 is the larger count but out of 50, so its rate is lower.
```

Build log entry to look for: ML09.ipynb with mean, median and range for [10, 12, 14, 16] before and after adding 100, and the two rates.

Misconceptions to listen for

- The median and mean are always the same.
- A bigger count means a bigger rate.
- Range tells me what is typical.

Questions to ask, with expected answers

- When is the mean a poor summary? Expected: When outliers pull it away from where most values sit.
- What is the percentage form of 0.5? Expected: 50%.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Typical value and spread: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, One extreme value and a proportion: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Typical value and spread): Find the 25th percentile of units.
2. Modify, Block B (One extreme value and a proportion): Replace 100 with 1000 in bigger and print the mean and the median.
3. Make: Compare 12 successes out of 20 attempts with 15 out of 50. Calculate each percentage and explain why the larger count does not have to mean the better rate.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: When is the mean a poor summary? Expected: When outliers pull it away from where most values sit.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML10: Cleaning without hiding evidence

Private. Keep this file out of the student folder.

Objective. By the end: you can count missing values and duplicates, remove an exact duplicate on a copy, and save a cleaning log.

Question of the day. What is wrong with this table, and what did I change?

> **WHERE THIS IS USED**
> Clinics, schools and banks collect data typed by busy people, so duplicates and blanks are normal. Data scientists clean the table and write down every change they make. What would you check first in a messy register?

Prerequisite: ML09 Mean, median, range and proportions.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Block A. Count the problems

A tiny table has two sets of trouble. Count them before you change anything.

Ask for the prediction first: Predict how many missing prices and how many duplicate rows the raw table has.

```
import pandas as pd
raw = pd.DataFrame({"id": [1, 2, 2, 3], "price": [1000, None, None, 3000]})
missing = raw["price"].isna().sum()
dupes = raw.duplicated().sum()
clean = raw.drop_duplicates().copy()
print(missing, dupes, len(clean))
print(clean["price"].mean())
```

Correct result: The cell prints 2, 1 and 3, then 2000.0. isna marks each blank as True and sum counts them. duplicated marks the repeated row and drop_duplicates removes it. copy makes a separate table, so raw stays unchanged. The mean of 2000.0 uses the two prices that exist and ignores the missing one.

Investigate, with answers:

- Why do we work on a copy? Answer: The raw table is the evidence. If a change is wrong we can start again from it.

- Why not replace the missing price with 0? Answer: Zero means a free item. A missing price is unknown, and zero would pull the mean down.

Modify: Print how many prices are still missing in clean. Answer: One price is still missing.

```
still_missing = clean["price"].isna().sum()
```

Block B. Save a cleaning log

Write down what you found as a small table and save it as a file.

Ask for the prediction first: Predict how many rows and columns the saved log has after you read it back.

```
log = pd.DataFrame({"step": ["raw rows", "duplicate removed", "price still missing"],
    "count": [4, 1, 1]})
log.to_csv("cleaning_log.csv", index=False)
log_back = pd.read_csv("cleaning_log.csv")
print(log_back)
print(log_back.shape)
```

Correct result: The log has 3 rows and 2 columns. to_csv writes the table into cleaning_log.csv. index=False stops pandas adding a column of row numbers. Reading the file back proves that it saved. The log lists the raw row count and each change, so a reader can repeat the cleaning.

Investigate, with answers:

- What would the log look like with index=False left out? Answer: The file would carry an extra unnamed column of row numbers.
- Who is the cleaning log for? Answer: Anyone who needs to check or repeat your work, including you next week.

Modify: Add the step final rows with a count of 3 and save again. Answer: The log has 4 rows.

```
log = pd.DataFrame({"step": ["raw rows", "duplicate removed", "price still missing",
    "final rows"], "count": [4, 1, 1, 3]})
log.to_csv("cleaning_log.csv", index=False)
log_back = pd.read_csv("cleaning_log.csv")
```

Make: reference

```
import pandas as pd
my_log = pd.DataFrame({"step": ["raw rows", "duplicate removed", "price missing"],
    "count": [4, 1, 1]})
my_log.to_csv("my_log.csv", index=False)
back = pd.read_csv("my_log.csv")
print(back)
# Zero would say the item was free. A missing price means the price is unknown.
```

Build log entry to look for: ML10.ipynb with the counts 2 missing, 1 duplicate, the saved log, and the explanation about zero.

Misconceptions to listen for

- Missing values should be filled with 0.

- Two equal rows are always a mistake.
- Cleaning means changing the original file.

Questions to ask, with expected answers

- Why can two equal rows be genuine? Expected: Two customers can buy the same item at the same price on the same day.
- What is the first thing to do with a messy file? Expected: Keep the raw copy, then count the problems.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Count the problems: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, Save a cleaning log: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Count the problems): Print how many prices are still missing in clean.
2. Modify, Block B (Save a cleaning log): Add the step final rows with a count of 3 and save again.
3. Make: Write a cleaning log for this table: four raw rows, one removed duplicate and one remaining missing price. Save it as my_log.csv and read it back. Explain why replacing the missing price with 0 would change its meaning.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: Why can two equal rows be genuine? Expected: Two customers can buy the same item at the same price on the same day.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML11: Charts that show what the numbers say

Private. Keep this file out of the student folder.

Objective. By the end: you can draw a labelled bar chart, a histogram and a box plot, and write one finding and one limitation.

Question of the day. How do I show which item earns most, and how sales vary, in a picture?

> **WHERE THIS IS USED**
> Weather forecasts, football stats and mobile money summaries all use charts so people can see a pattern fast. A good chart makes one clear point. What is the one point you want your chart to make?

Prerequisite: ML10 Cleaning without hiding evidence.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Block A. A bar chart of revenue

Chart the revenue of each item from the practice shop. Label the axis with its unit.

Ask for the prediction first: Predict which bar will be tallest and roughly how tall.

```
import matplotlib.pyplot as plt
import pandas as pd
shop = pd.read_csv("camp_shop.csv")
shop["revenue_ugx"] = shop["units"] * shop["unit_price_ugx"]
totals = shop.groupby("item")["revenue_ugx"].sum()
plt.bar(totals.index, totals.values)
plt.ylabel("Revenue (UGX)")
plt.title("Revenue by item in the practice shop")
plt.ylim(bottom=0)
plt.show()
```

Correct result: Three bars appear. Book is tallest at 297000, then Juice at 150000 and Pen at 103000. plt is the chart tool from matplotlib. ylabel and title add words, and ylim(bottom=0) starts the axis at zero so bar heights are fair. One finding is: Book earns the most, UGX 297000 of 550000. One limitation is: the data is invented and has no costs.

Investigate, with answers:

- Why must a bar chart start at zero? Answer: A bar's height should match its size. A cut-off axis exaggerates small differences.
- Does the chart tell us why Book earns most? Answer: No. It shows the totals only; the reason is a separate question.

Modify: Chart total units instead of revenue. Answer: Pen is now tallest, with 103 units.

```
units_by_item = shop.groupby("item")["units"].sum()
plt.bar(units_by_item.index, units_by_item.values)
plt.ylabel("Units sold")
plt.show()
```

Block B. A histogram and a box plot

seaborn draws statistical charts and builds on matplotlib. Draw how units vary and how they differ by item.

Ask for the prediction first: Predict which item has the lowest median units.

```
import seaborn as sns
sns.histplot(data=shop, x="units", discrete=True)
plt.title("Units per sale")
plt.show()
sns.boxplot(data=shop, x="item", y="units")
plt.title("Units per sale for each item")
plt.show()
medians = shop.groupby("item")["units"].median()
```

Correct result: The histogram counts sales for each number of units and shows how they spread from 1 to 8. The box plot shows one box per item with a line at its median. Juice has the lowest median, 3.5 units, while Book has 5.0 and Pen 5.5. seaborn plots read straight from the table, which saves writing the grouping yourself.

Investigate, with answers:

- What does the line inside a box mean? Answer: The median of that item's units.
- Which chart shows how many sales had exactly 3 units? Answer: The histogram.

Modify: Change the histogram so it uses unit_price_ugx instead of units. Answer: Three tall bars appear, one at each price.

```
sns.histplot(data=shop, x="unit_price_ugx")
plt.show()
```

Make: reference

```
import matplotlib.pyplot as plt
items = ["Book", "Pen", "Juice"]
heights = [6, 9, 4]
plt.bar(items, heights)
plt.ylabel("Units sold")
plt.title("Practice shop units sold")
plt.ylim(bottom=0)
plt.show()
# Finding: Pen sells 9 units, 3 more than Book. Limitation: the table is small and
    invented. Daily rainfall suits a line chart because the days are in order.
```

Build log entry to look for: ML11.ipynb with a labelled bar chart, one finding with a number, one limitation, and the rainfall answer.

Misconceptions to listen for

- A chart proves why something happened.
- The axis can start anywhere.
- A box plot shows every value.

Questions to ask, with expected answers

- What is the first thing to look at on a chart? Expected: The title and the axis labels with their units.
- What can a bar chart never show you about cause? Expected: Anything. It shows totals and nothing about reasons.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, A bar chart of revenue: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, A histogram and a box plot: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (A bar chart of revenue): Chart total units instead of revenue.
2. Modify, Block B (A histogram and a box plot): Change the histogram so it uses unit_price_ugx instead of units.

3. Make: Make the same bar chart for the values 6, 9 and 4 for Book, Pen and Juice. Write one finding with a number and one limitation. Say which chart type you would use for daily rainfall, and why.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What is the first thing to look at on a chart? Expected: The title and the axis labels with their units.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML12: Show your first chart and choose your project question

Private. Keep this file out of the student folder.

Objective. By the end: you can load a table, check it, total it, chart it, write two findings and a limitation, and ask one prediction question a table could answer.

Question of the day. Can I take a table from a file to a labelled chart and a careful finding on my own?

> **WHERE THIS IS USED**
> Every data job starts with one clear question and a table. A coffee trader, a football analyst and a clinic manager each begin that way. Which question would you like your own project to answer?

Prerequisite: ML11 Charts that show what the numbers say.

The 180 minutes

Minutes	Activity
10	recall
20	brief and worked skeleton
15	break
100	first-chart work
20	project question
15	save and exit check

Where it overruns: Most time goes on the independent chart work. Keep the skeleton short and let students use their own wording.

Block A. The first-chart skeleton

Run the skeleton, then repeat each step in your own notebook, ML12.ipynb, with your own wording.

Ask for the prediction first: Predict the row count, the total revenue and the best-selling item by revenue.

```
import pandas as pd
import matplotlib.pyplot as plt
shop = pd.read_csv("camp_shop.csv")
missing = shop.isna().sum().sum()
shop["revenue_ugx"] = shop["units"] * shop["unit_price_ugx"]
by_item = shop.groupby("item")["revenue_ugx"].sum()
print(shop.shape[0], missing, by_item.sum())
assert by_item.sum() == shop["revenue_ugx"].sum()
plt.bar(by_item.index, by_item.values)
plt.ylabel("Revenue (UGX)")
plt.title("Revenue by item, practice shop")
plt.show()
```

Correct result: The skeleton prints 60 rows, 0 missing cells and a total revenue of 550000. The assert passes silently because the item totals add up to the overall total. The chart shows Book as the tallest bar. Your own notebook needs the same steps with your two findings, one limitation, a missing-value check and a clean rerun from a restarted kernel.

Investigate, with answers:

- What does the passing assert prove? Answer: That the grouped totals and the overall total agree.
- Why record that there were no missing values? Answer: A check that finds nothing is still evidence that you looked.

Modify: Add a finding with a number and one limitation as text cells under the chart. Answer: For example: Book earns UGX 297000, about 54% of revenue. The data is invented and has no costs.

```
book_share = by_item['Book'] / by_item.sum()
```

Block B. Write your project question

Start from the topic on your progress map. In the last 20 minutes choose the supplied table that fits it best and write three sentences: the question, the dataset and the target. Model building starts next week.

Ask for the prediction first: Before you look at the datasets again, write the question you would most like a model to answer.

Correct result: Your three sentences name a question a table can answer, one supplied dataset (synthetic_homes.csv, synthetic_fares.csv or the iris flowers), and the target to predict. Inputs must be known before the target is. Good: predict monthly rent from area and rooms. Weak: explain why rents are high.

Investigate, with answers:

- Which column is the target in synthetic_fares.csv? Answer: fare_ugx, the price of the trip.
- Why does a prediction question need inputs known beforehand? Answer: A model cannot use information that only exists after the answer.

Modify: Rewrite a question that asks why into one that asks for a prediction. Answer: Why are rents high becomes: can area and rooms predict monthly rent?

Make: reference

```
import pandas as pd
import matplotlib.pyplot as plt
shop = pd.read_csv("camp_shop.csv")
missing = shop.isna().sum().sum()
shop["revenue_ugx"] = shop["units"] * shop["unit_price_ugx"]
by_item = shop.groupby("item")["revenue_ugx"].sum()
total_units = shop["units"].sum()
print(len(shop), missing, total_units, by_item.to_dict())
```

Build log entry to look for: ML12.ipynb: totals, one labelled chart, two findings, one limitation, a missing-value check, a restarted-kernel run, and a three-sentence project question.

Misconceptions to listen for

- Showing my first chart needs a new technique.
- A prediction question can be any question.
- No missing values means no check is needed.

Questions to ask, with expected answers

- What do you do first if the totals disagree? Expected: Compare row counts at each step to find where rows changed.
- What makes a project question suitable? Expected: A table can answer it, a target exists, and its inputs are known beforehand.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, The first-chart skeleton: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, Write your project question: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (The first-chart skeleton): Add a finding with a number and one limitation as text cells under the chart.
2. Modify, Block B (Write your project question): Rewrite a question that asks why into one that asks for a prediction.
3. Make: In ML12.ipynb report the row count, the revenue by item, one chart, two findings and one limitation, and record your missing-value check. Run all cells from a restarted kernel and save. Add your three-sentence project question. Start from the topic on your progress map and pick the supplied table that fits it best. If none fits, say so; real public data is used only after you and the instructor have read its licence.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What do you do first if the totals disagree? Expected: Compare row counts at each step to find where rows changed.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML13: Features, targets and an honest baseline

Private. Keep this file out of the student folder.

Objective. By the end: you can name the features and the target in a table, and calculate a baseline error that any model must beat.

Question of the day. What did the session 1 model learn from, and how well would a plain guess do?

> **WHERE THIS IS USED**
> Banks and mobile lenders use models to flag unusual payments, and they first ask whether the model beats a plain rule. That plain rule is the baseline. What plain guess would you make for tomorrow's weather?

Prerequisite: ML12 Show your first chart and choose your project question.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Block A. The baseline guess

A baseline uses no features at all. It guesses the average of the training targets every time.

Ask for the prediction first: Training prices are 2000, 4000 and 6000. Predict the baseline guess and its error when the real price is 5000.

```
train_prices = [2000, 4000, 6000]
baseline = sum(train_prices) / len(train_prices)
actual = 5000
error = abs(actual - baseline)
print(baseline, error)
```

Correct result: The cell prints 4000.0 and 1000.0. The training mean 4000.0 is the baseline guess. abs removes the minus sign, so the error is the distance between the guess and the real value, 1000 shillings. A model is worth using only if its errors on unseen examples are smaller than the baseline's.

Investigate, with answers:

- Why must the baseline use only training prices? Answer: Test prices are answers the model has not seen. Using them would be a form of leakage.
- What would the baseline guess be for every new price? Answer: 4000.0, the same number each time.

Modify: Change the actual price to 3000 and recalculate the error. Answer: The error is 1000.0 again, because 3000 is also 1000 below 4000.

```
actual = 3000
error = abs(actual - baseline)
```

Block B. Open the session 1 model

The model from session 1 had one feature and one target. Print what it learned.

Ask for the prediction first: Name the feature and the target in this code before you run it.

```
import pandas as pd
from sklearn.linear_model import LinearRegression
homes = pd.read_csv("synthetic_homes.csv")
X = homes[["area_m2"]]
y = homes["rent_ugx"]
model = LinearRegression().fit(X, y)
slope = model.coef_[0]
intercept = model.intercept_
print(round(slope), round(intercept))
```

Correct result: The cell prints 2996 and 159436. The feature is area_m2, kept in X. The target is rent_ugx, kept in y. fit looked at all 120 homes and found a line: rent is about 159436 shillings plus 2996 shillings for each square metre. That model was scored on nothing at all. Judging it on the same homes it learned from would flatter it, so next session we keep some homes back.

Investigate, with answers:

- Which of these is the feature and which is the target? Answer: area_m2 is the feature, rent_ugx is the target.
- Why is it unfair to test the model on the same 120 homes? Answer: It has already seen their answers. A fair check needs homes it has not seen.

Modify: Add rooms as a second feature and print the two slopes. Answer: About 2992 shillings per square metre and 19345 shillings per room.

```
X2 = homes[["area_m2", "rooms"]]
model2 = LinearRegression().fit(X2, y)
slopes = [round(model2.coef_[0]), round(model2.coef_[1])]
```

Make: reference

```
train_values = [2000, 4000, 6000]
baseline = sum(train_values) / len(train_values)
error = abs(5000 - baseline)
print(baseline, error)
# 4000.0 and 1000.0. The final payment amount would give away the selling price: leakage.
```

Build log entry to look for: ML13.ipynb with baseline 4000.0, error 1000.0, and one named leaked feature.

Misconceptions to listen for

- The model should be tested on the homes it learned from.
- The baseline is a poor attempt at a model.
- Any column in the table is a fair input.

Questions to ask, with expected answers

- What are the feature and target in the session 1 model? Expected: Feature: area_m2. Target: rent_ugx.
- What counts as leakage in a rent model? Expected: Any column only known after the rent is agreed, such as the first month's payment received.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, The baseline guess: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, Open the session 1 model: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (The baseline guess): Change the actual price to 3000 and recalculate the error.
2. Modify, Block B (Open the session 1 model): Add rooms as a second feature and print the two slopes.
3. Make: For training values [2000, 4000, 6000], find the baseline guess for a new item. If its real value is 5000, calculate the absolute error. Name one leaked feature if you were predicting a selling price.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What are the feature and target in the session 1 model? Expected: Feature: area_m2. Target: rent_ugx.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML14: Predicting a number with a line

Private. Keep this file out of the student folder.

Objective. By the end: you can guess a line by hand, measure its error, and compare it with the line scikit-learn fits.

Question of the day. How close does a straight line get to the rents of eight homes?

> **WHERE THIS IS USED**
> Rents, fares and crop prices often rise in a steady way with size, distance or season, and a straight line is the simplest model of that. Ride-hailing apps show a fare estimate before the trip starts. How far can one line take you?

Prerequisite: ML13 Features, targets and an honest baseline.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Where it overruns: Draw the eight homes on a printed scatter plot first and let students sketch a line with a ruler before any code.

Block A. Guess a line, measure the error

Take the first eight homes. Guess that rent is 100000 plus 3000 for every square metre.

Ask for the prediction first: Predict whether the average error will be under UGX 10000 or over UGX 50000.

```
import pandas as pd
homes = pd.read_csv("synthetic_homes.csv").head(8)
guess = 100000 + 3000 * homes["area_m2"]
errors = (homes["rent_ugx"] - guess).abs()
print(errors.mean())
```

Correct result: The cell prints 56282.25. Your guess is a straight line with intercept 100000 and slope 3000 shillings per square metre. On average it misses the real rent by about UGX 56282. The line ignores rooms, and the intercept is too low, so most guesses are too small.

Investigate, with answers:

- What does a slope of 3000 mean in words? Answer: Every extra square metre adds UGX 3000 to the guess.

- What would you change to lower the error? Answer: Raise the intercept. Most guesses are below the real rent.

Modify: Try an intercept of 160000 and print the average error. Answer: The average error falls to about UGX 21822.

```
guess2 = 160000 + 3000 * homes["area_m2"]
error2 = (homes["rent_ugx"] - guess2).abs().mean()
```

Block B. Let scikit-learn fit the line

Ask LinearRegression to find the best slope and intercept for the same eight homes. Compare its error with the baseline.

Ask for the prediction first: Predict whether the fitted line's error is smaller or larger than your hand-guessed 56282.

```
from sklearn.linear_model import LinearRegression
X = homes[["area_m2"]]
y = homes["rent_ugx"]
model = LinearRegression().fit(X, y)
fit_error = (y - model.predict(X)).abs().mean()
baseline_error = (y - y.mean()).abs().mean()
print(round(model.coef_[0]), round(model.intercept_), round(fit_error),
    round(baseline_error))
```

Correct result: The cell prints 3221, 135838, 20077 and 109399. The fitted line has slope 3221 and intercept 135838 and misses by about UGX 20077 on these eight homes. Guessing the mean every time would miss by about UGX 109399. fit found the line, and predict will use it on a new input. These errors are measured on the eight homes the line learned from, so they look better than they will on new homes.

Investigate, with answers:

- Is a smaller error on the training homes proof that the line is good? Answer: No. The line has seen these homes. A fair check uses new homes, which is next session.
- How much better is the fitted line than the baseline? Answer: Its error is about a fifth of the baseline's.

Modify: Predict the rent for a home of 100 square metres. Answer: About UGX 457910.

```
pred100 = model.predict(pd.DataFrame({"area_m2": [100]}))[0]
```

Make: reference

```
from sklearn.linear_model import LinearRegression
X = [[10], [20], [30], [40]]
y = [100, 200, 300, 400]
model = LinearRegression().fit(X, y)
p35 = model.predict([[35]])[0]
print(p35)
# About 350. X holds inputs, y the answers. fit learns the line and predict applies it.
    1000 is far outside the inputs it saw.
```

Build log entry to look for: ML14.ipynb with the prediction 350, the four definitions in the student's own words, and the reason 1000 is risky.

Misconceptions to listen for

- The fitted line must pass through every point.
- A smaller training error always means a better model.
- I can predict anywhere on the line safely.

Questions to ask, with expected answers

- What does the slope of 3221 mean here? Expected: On these eight homes each extra square metre adds about UGX 3221 of rent.
- Why is the data on the printed scatter plot useful? Expected: It lets you see where a hand-drawn line sits among the points.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Guess a line, measure the error: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, Let scikit-learn fit the line: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Guess a line, measure the error): Try an intercept of 160000 and print the average error.
2. Modify, Block B (Let scikit-learn fit the line): Predict the rent for a home of 100 square metres.
3. Make: Use X = [[10], [20], [30], [40]] and y = [100, 200, 300, 400]. Fit a line and predict at 35. Explain in your own words what X, y, fit and predict mean, and why predicting at 1000 is risky.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What does the slope of 3221 mean here? Expected: On these eight homes each extra square metre adds about UGX 3221 of rent.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML15: A complete regression experiment

Private. Keep this file out of the student folder.

Objective. By the end: you can split data, fit a model and a baseline on the training part, and compare their errors on held-out homes.

Question of the day. Does the rent model beat a plain guess on homes it has never seen?

> **WHERE THIS IS USED**
> A model can look perfect on the homes it learned from and still fail on a new one. Teams that build tools for lending, fares or weather test on cases kept aside. Would you trust a model that was only tested on its own homework?

Prerequisite: ML14 Predicting a number with a line.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Where it overruns: The unpacking of four names into one line is the hardest part. Draw the four boxes on the board before running.

Block A. Split the homes

Keep a quarter of the 120 homes back. The model must never see them while it learns.

Ask for the prediction first: Predict how many homes go to training and how many to testing.

```
import pandas as pd
from sklearn.model_selection import train_test_split
homes = pd.read_csv("synthetic_homes.csv")
X = homes[["area_m2", "rooms"]]
y = homes["rent_ugx"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=7)
print(len(X_train), len(X_test))
```

Correct result: The cell prints 90 and 30. The four names on the left receive training inputs, test inputs, training targets and test targets, in that order. random_state=7 fixes the shuffle so everyone gets the same split.

Investigate, with answers:

- Why do we shuffle before splitting? Answer: The file may be in an order that puts similar homes together. A shuffle gives both parts a fair mix.
- Which of the four names would hold the answers you must not peek at? Answer: y_test.

Modify: Change test_size to 0.2 and run the split again. Answer: 96 homes train and 24 test.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=7)
```

Block B. Baseline against model

Fit a mean-guess baseline and a line on the 90 training homes. Measure both on the 30 held-out homes.

Ask for the prediction first: Predict whether the model's average miss will be smaller than the baseline's, and by roughly how much.

```
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
baseline = DummyRegressor(strategy="mean").fit(X_train, y_train)
model = LinearRegression().fit(X_train, y_train)
baseline_mae = mean_absolute_error(y_test, baseline.predict(X_test))
model_mae = mean_absolute_error(y_test, model.predict(X_test))
print(round(baseline_mae), round(model_mae))
```

Correct result: The cell prints 110169 and 11889. The baseline misses by about UGX 110169 and the model by about UGX 11889, so on these 30 held-out homes the model is far better. Mean absolute error averages the misses and uses the unit of the target, shillings. The model wins by this much because the homes were generated from a formula, which makes them easy to learn. Real data would not be so kind.

Investigate, with answers:

- What does a mean absolute error of 11889 mean in words? Answer: On average the model's rent estimate is about UGX 11889 away from the real rent.
- Why is this result no evidence about real rents? Answer: The homes are invented from a neat formula, so any line fits them well.

Modify: Fit a model using only area_m2 and print its error. Answer: About UGX 28272, worse than with rooms too.

```
area_only = LinearRegression().fit(X_train[["area_m2"]], y_train)
area_mae = mean_absolute_error(y_test, area_only.predict(X_test[["area_m2"]]))
```

Make: reference

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
homes = pd.read_csv("synthetic_homes.csv")
X = homes[["area_m2", "rooms"]]
y = homes["rent_ugx"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=7)
baseline_mae = mean_absolute_error(y_test, DummyRegressor(strategy="mean").fit(X_train,
    y_train).predict(X_test))
model_mae = mean_absolute_error(y_test, LinearRegression().fit(X_train,
    y_train).predict(X_test))
print(round(baseline_mae), round(model_mae))
```

Build log entry to look for: ML15.ipynb with both errors, the sentence that the model beats the baseline on held-out homes, and the split unchanged.

Misconceptions to listen for

- Changing random_state until the score looks good is fine.
- The model is tested on the training homes.
- A low error on invented data proves real accuracy.

Questions to ask, with expected answers

- Why report the baseline error next to the model error? Expected: Without it nobody knows whether the model is better than a plain guess.
- What is held out? Expected: Rows the model never saw while it learned.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Split the homes: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, Baseline against model: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Split the homes): Change test_size to 0.2 and run the split again.
2. Modify, Block B (Baseline against model): Fit a model using only area_m2 and print its error.

3. Make: Run the experiment, record the two errors, and say whether the model beats the baseline on the held-out homes. Keep the split as it is. Do not keep changing random_state to get a better score.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: Why report the baseline error next to the model error? Expected: Without it nobody knows whether the model is better than a plain guess.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML16: Predicting a category with a decision tree

Private. Keep this file out of the student folder.

Objective. By the end: you can train a small decision tree on iris, read its first question, and compare its accuracy with a baseline and with the hand rule from session 4.

Question of the day. Does a tree that learned its questions beat my hand-written flower rule?

> **WHERE THIS IS USED**
> Doctors and farm advisers sort cases into groups by asking questions in order: is there a fever, are the leaves spotted? A decision tree learns which questions to ask. Can your tree out-guess your own rule?

Prerequisite: ML15 A complete regression experiment.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Block A. Train a tree and draw it

Split the 150 iris flowers, train a tree that may ask two questions in a row, and draw it.

Ask for the prediction first: Predict whether the tree's accuracy will be above 85% and whether the majority baseline will be above 55%.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,
    test_size=0.25, random_state=7)
tree = DecisionTreeClassifier(max_depth=2, random_state=7).fit(X_train, y_train)
baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
tree_acc = accuracy_score(y_test, tree.predict(X_test))
base_acc = accuracy_score(y_test, baseline.predict(X_test))
print(round(tree_acc, 3), round(base_acc, 3))
plot_tree(tree, feature_names=iris.feature_names, class_names=list(iris.target_names))
plt.show()
```

Correct result: The tree scores above 0.85 and the majority baseline scores about a third, because the three species are equally common. plot_tree draws the questions: the top box asks about a petal measurement, the same kind of question as your hand rule. The classes are coded 0, 1 and 2 for setosa, versicolor and virginica. The codes name categories, so species 2 is not twice species 1. Accuracy is correct predictions divided by all predictions.

Investigate, with answers:

- What does the top box of the drawn tree ask? Answer: A yes or no question about a petal measurement. Read it aloud.
- Why does the baseline score near one third? Answer: Each species has about a third of the flowers, so always naming one is right about a third of the time.
- Why is species code 2 not twice species code 1? Answer: The codes are labels. They carry no amounts.

Modify: Change max_depth to 1 and run again. Answer: The tree has one question and scores lower, about two thirds.

```
tree1 = DecisionTreeClassifier(max_depth=1, random_state=7).fit(X_train, y_train)
acc1 = accuracy_score(y_test, tree1.predict(X_test))
```

Block B. Back to the hand rule

Score your session 4 rule on all 150 flowers. Petal length is column 2 of the data. The rule says setosa below 2.5, versicolor below 4.8 and virginica otherwise.

Ask for the prediction first: Predict whether your hand rule will score above or below the tree.

```
lengths = iris.data[:, 2]
guess = (lengths >= 2.5).astype(int) + (lengths >= 4.8).astype(int)
hand_acc = (guess == iris.target).mean()
print(round(hand_acc, 3))
```

Correct result: The hand rule scores above 0.9 on all 150 flowers. Each comparison gives True or False for every flower and astype(int) turns them into 1 and 0. Adding the two gives 0 for setosa, 1 for versicolor and 2 for virginica. Your rule and the tree use the same idea. The tree chose its question and cut-off by itself, and you chose yours by looking at eight flowers.

Investigate, with answers:

- Why does the hand rule use only petal length? Answer: That was the one measurement you looked at. The tree may use any of the four.
- Whose score is fairer: the hand rule on all 150 flowers, or the tree on the 38 held-out flowers of a 112 and 38 split? Answer: Neither is exactly fair. Your rule was designed with a few of these flowers in view, and the tree's test set is small.

Modify: Change 4.8 to 5.0 in the rule and score it again. Answer: The score changes a little. Check whether it rises or falls.

```
guess5 = (lengths >= 2.5).astype(int) + (lengths >= 5.0).astype(int)
acc5 = (guess5 == iris.target).mean()
```

Make: reference

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,
    test_size=0.25, random_state=7)
tree = DecisionTreeClassifier(max_depth=2, random_state=7).fit(X_train, y_train)
percent = accuracy_score(y_test, tree.predict(X_test)) * 100
print(round(percent, 1))
# Target: species. One input: petal length. The codes are labels that carry no amounts.
```

Build log entry to look for: ML16.ipynb with both accuracies as percentages, the target and one input named, and the sentence about codes.

Misconceptions to listen for

- Species code 2 is twice species code 1.
- A tree always beats a hand-written rule.
- An accuracy of 100% on training flowers means the tree is perfect.

Questions to ask, with expected answers

- What does max_depth limit? Expected: How many questions in a row the tree may ask.
- Why print the baseline accuracy? Expected: To show how much better than guessing the tree is.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Train a tree and draw it: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, Back to the hand rule: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Train a tree and draw it): Change max_depth to 1 and run again.
2. Modify, Block B (Back to the hand rule): Change 4.8 to 5.0 in the rule and score it again.
3. Make: Convert both accuracies to percentages. Name the target and one input. Explain why species code 2 is not twice species code 1.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What does max_depth limit? Expected: How many questions in a row the tree may ask.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML17: Mistakes, overfitting and fair comparisons

Private. Keep this file out of the student folder.

Objective. By the end: you can read a confusion matrix, spot an overfitted tree from its training and test scores, and explain why the held-back set must stay untouched until the end.

Question of the day. When is a very high score a warning sign?

> **WHERE THIS IS USED**
> A fraud alert that blocks a real payment upsets a customer, and one that misses fraud costs money. Teams count both kinds of mistake. When would a very high score make you suspicious?

Prerequisite: ML16 Predicting a category with a decision tree.

The 180 minutes

Minutes	Activity
10	Recall: retrieve yesterday, three questions on paper
50	Block A: predict, run, investigate, modify
15	Break away from screens
50	Block B: predict, run, investigate, modify
40	Make: the independent task and the project journal
15	Exit ticket, save, and an individual check with each learner

Block A. Read a confusion matrix

A model guessed five yes or no answers. Count the matches.

Ask for the prediction first: The matrix will have two rows and two columns. Predict which cells hold the correct answers.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
actual = [0, 0, 1, 1, 1]
predicted = [0, 1, 1, 0, 1]
matrix = confusion_matrix(actual, predicted, labels=[0, 1])
print(matrix)
ConfusionMatrixDisplay(matrix, display_labels=["no", "yes"]).plot()
plt.show()
```

Correct result: The cell prints [[1 1] [1 2]]. Rows are the actual answers and columns are the predicted answers. The top-left cell is an actual no predicted no, and the bottom-right is an actual yes predicted yes. The correct answers are on the diagonal: 1 plus 2 is 3 out of 5. ConfusionMatrixDisplay draws the table as a picture.

Investigate, with answers:

- Which cell counts an actual yes that the model called no? Answer: The bottom-left cell. It holds 1.
- Why read the axis labels before the numbers? Answer: If you mix up rows and columns you will swap the two kinds of mistake.

Modify: Calculate the accuracy from the matrix. Answer: 3 divided by 5 is 0.6, or 60%.

```
accuracy = (matrix[0, 0] + matrix[1, 1]) / matrix.sum()
```

Block B. The tree that remembered

Train trees that may ask more and more questions, and print their training and test scores.

Ask for the prediction first: Predict which tree scores 100% on the training flowers.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,
    test_size=0.25, random_state=7)
for depth in [1, 2, 3, None]:
    model = DecisionTreeClassifier(max_depth=depth, random_state=7).fit(X_train, y_train)
    print(depth, round(model.score(X_train, y_train), 3), round(model.score(X_test,
        y_test), 3))
gap = model.score(X_train, y_train) - model.score(X_test, y_test)
```

Correct result: Each line prints the depth, the training score and the test score. The tree with depth None may ask as many questions as it likes and scores 1.0 on the training flowers, because it memorised them. Its test score does not rise with it. That gap between training and test is the sign of overfitting. score gives the accuracy.

Investigate, with answers:

- Why is 100% on training flowers a warning sign? Answer: The tree may have memorised the flowers. Only the test score says how it does on new ones.
- Can we use the test score to pick the best depth? Answer: No. A choice made by looking at test scores uses those rows to tune the model, and the held-back set then gives no honest result.

Modify: Print the gap between the last tree's training and test scores. Answer: The gap is the training score minus the test score; it is zero or more.

```
gap2 = model.score(X_train, y_train) - model.score(X_test, y_test)
```

Make: reference

```
from sklearn.metrics import confusion_matrix
matrix = confusion_matrix([0, 0, 1, 1, 1], [0, 1, 1, 0, 1], labels=[0, 1])
right = matrix[0, 0] + matrix[1, 1]
wrong = matrix.sum() - right
print(right, wrong)
# 3 right, 2 wrong. If you tune against the held-back set, it has become a validation set
    and a new held-back set is needed.
```

Build log entry to look for: ML17.ipynb with the labelled matrix, 3 right and 2 wrong, and the explanation about tuning on the test.

Misconceptions to listen for

- A higher training score always means a better model.
- I can try settings on the held-back set until I like the result.
- A small test set gives stable scores.

Questions to ask, with expected answers

- What does the diagonal of a confusion matrix hold? Expected: The correct predictions.
- What changes if you choose a depth by looking at test scores? Expected: The test is no longer unseen, so it cannot give an honest final score.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Read a confusion matrix: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, The tree that remembered: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Read a confusion matrix): Calculate the accuracy from the matrix.
2. Modify, Block B (The tree that remembered): Print the gap between the last tree's training and test scores.
3. Make: Draw the 2 by 2 matrix for the toy answers and label both axes. Count the right and the wrong predictions. Explain why trying many settings against the held-back set weakens it.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What does the diagonal of a confusion matrix hold? Expected: The correct predictions.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML18: A fair comparison: does your model beat the plain guess?

Private. Keep this file out of the student folder.

Objective. By the end: you can split data into training, validation and held-back parts, choose a model on validation only, and report one held-back result with its limits.

Question of the day. Which tree depth is better, and does it beat the plain guess on the held-back flowers?

> **WHERE THIS IS USED**
> A football coach tests two formations on matches the team has not practised for. You do the same with two tree depths. Would you pick a formation after one lucky match?

Prerequisite: ML17 Mistakes, overfitting and fair comparisons.

The 180 minutes

Minutes	Activity
10	recall
25	split and plan
15	break
100	comparison work
15	explanation of the decision
15	save and exit check

Where it overruns: Keeping the held-back set untouched is the point. Check each learner's order of steps.

Block A. A three-way split

First hold back 20% of the flowers as the held-back set, then take a quarter of what remains for validation. stratify keeps each species equally represented in every part.

Ask for the prediction first: Predict the three sizes. They should add up to 150.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
X, y = load_iris(return_X_y=True)
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.2, random_state=7,
    stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.25,
    random_state=7, stratify=y_dev)
print(len(X_train), len(X_val), len(X_test))
```

Correct result: The cell prints 90, 30 and 30. stratify=y means each species has the same share in every part: 10 of each species in validation and in the held-back set. The held-back set is now set aside and

must not be used to make any choice.

Investigate, with answers:

- Why are the three parts 60%, 20% and 20%? Answer: 20% is held back first. A quarter of the other 80% is 20% of the total, which leaves 60%.
- What does stratify do that a plain shuffle does not? Answer: It makes sure each part has the same species mix, so a small part is not accidentally missing one.

Modify: Print how many flowers of species 0 are in the validation part. Answer: Ten.

```
n_setosa_val = int((y_val == 0).sum())
```

Block B. Choose on validation, test once

Compare a tree of depth 1 and one of depth 3 on validation only. Freeze the winner. Then use the held-back set once.

Ask for the prediction first: Predict which depth wins on validation. Write it down before you run.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
val_baseline = accuracy_score(y_val, baseline.predict(X_val))
tree1 = DecisionTreeClassifier(max_depth=1, random_state=7).fit(X_train, y_train)
tree3 = DecisionTreeClassifier(max_depth=3, random_state=7).fit(X_train, y_train)
score1 = accuracy_score(y_val, tree1.predict(X_val))
score3 = accuracy_score(y_val, tree3.predict(X_val))
if score3 > score1:
    chosen_depth = 3
    final_model = tree3
else:
    chosen_depth = 1
    final_model = tree1
final_acc = accuracy_score(y_test, final_model.predict(X_test))
print(val_baseline, score1, score3, chosen_depth, final_acc)
print(confusion_matrix(y_test, final_model.predict(X_test)))
```

Correct result: The baseline scores exactly one third on validation, because each species is a third. The deeper tree usually wins on validation. The choice is made from the validation scores and then frozen. final_acc is the one honest number: it comes from 30 flowers the choice never saw. The report needs the split sizes, the baseline, both validation scores, the chosen depth, the held-back accuracy, the confusion matrix and two limitations.

Investigate, with answers:

- Which scores did you use to choose the depth? Answer: The validation scores only.
- What is one limitation of a held-back set of 30 flowers? Answer: One flower is more than 3 points of accuracy, so the score is unstable.

Modify: Say what you would do if the held-back score came out lower than the validation score. Answer: Report it as it is. A lower final score is normal, and changing the choice now would use the held-back set twice.

Make: reference

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
X, y = load_iris(return_X_y=True)
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.2, random_state=7,
    stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.25,
    random_state=7, stratify=y_dev)
tree1 = DecisionTreeClassifier(max_depth=1, random_state=7).fit(X_train, y_train)
tree3 = DecisionTreeClassifier(max_depth=3, random_state=7).fit(X_train, y_train)
scores = [accuracy_score(y_val, tree1.predict(X_val)), accuracy_score(y_val,
    tree3.predict(X_val))]
print(len(X_train), len(X_val), len(X_test), scores)
```

Build log entry to look for: ML18.ipynb: split sizes 90, 30 and 30, baseline, two validation scores, the chosen depth, one held-back accuracy, a confusion matrix and two limitations.

Misconceptions to listen for

- The depth with the highest held-back accuracy is best in general.
- The held-back set can be used twice if the first result looks bad.
- A small test set gives a precise score.

Questions to ask, with expected answers

- Which score picks the depth? Expected: The validation score.
- Why report two limitations? Expected: The data is small and the test is only 30 flowers, so the result is uncertain.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, A three-way split: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, Choose on validation, test once: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (A three-way split): Print how many flowers of species 0 are in the validation part.

2. Modify, Block B (Choose on validation, test once): Say what you would do if the held-back score came out lower than the validation score.
3. Make: Save ML18.ipynb with: the split sizes, the baseline, two validation scores, the chosen depth, the held-back accuracy, the confusion matrix and two limitations. Explain your decision without claiming that the highest score proves one depth is best in general.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: Which score picks the depth? Expected: The validation score.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML19: Project data and a safe plan

Private. Keep this file out of the student folder.

Objective. By the end: you can describe a supplied dataset, spot a leakage column, and write a seven-heading project plan the instructor can approve.

Question of the day. Which dataset and which question can I answer with a fair comparison in two weeks?

> **WHERE THIS IS USED**
> Real projects start with a written plan: the question, the data, and the column that would give the answer away. Data teams also protect private information. Which question could your data answer fairly?

Prerequisite: ML18 A fair comparison: does your model beat the plain guess?.

The 180 minutes

Minutes	Activity
10	recall
40	data tour
15	break
35	leakage and the seven headings
60	writing PROJECT_PLAN.md and instructor approval
20	AI disclosure rules and exit check

Where it overruns: Instructor approval of each plan is the bottleneck. Visit learners in turn and ask them to finish the baseline heading first.

Block A. Tour the fares data

synthetic_fares.csv holds 150 invented boda-boda trips. Look at its shape and at how each column moves with the fare.

Ask for the prediction first: Predict which column moves most closely with fare_ugx.

```
import pandas as pd
fares = pd.read_csv("synthetic_fares.csv")
print(fares.shape)
corr = fares[["distance_km", "rush_hour", "raining", "tip_ugx",
    "fare_ugx"]].corr()["fare_ugx"].round(2)
print(corr)
```

Correct result: The table has 150 rows and 6 columns. Distance has a correlation of 0.76 with the fare, rain 0.12 and rush hour about minus 0.08. The tip column has the highest value, 0.88. A correlation near 1 means the two columns rise together. The tip rises with the fare because it is a share of the fare, and it is only known after the trip, so using it to predict the fare would be cheating. That makes tip_ugx a leakage column.

Investigate, with answers:

- Why can tip_ugx not be an input? Answer: It is known only after the ride, and it is calculated from the fare. The model would be given the answer.
- Which columns are sensible inputs? Answer: distance_km, rush_hour and raining. All three are known before the ride starts.

Modify: Print the correlation of the fare with the trip_id column. Answer: It is close to zero: trip_id is only a label.

```
id_corr = fares["trip_id"].corr(fares["fare_ugx"])
```

Block B. A plan needs seven headings

The plan has seven headings: question, dataset, target, inputs, baseline, evaluation and limits. Write one sentence under each. Begin with the topic on your progress map and the supplied dataset that fits it best.

Ask for the prediction first: Write the target and two inputs for your project before looking at the example.

Correct result: An example plan: question, can distance and rush hour predict a boda-boda fare; dataset, synthetic_fares.csv; target, fare_ugx; inputs, distance_km, rush_hour and raining; baseline, the mean fare of the training trips; evaluation, mean absolute error in shillings on a held-back set; limits, invented data and 150 trips. The options are synthetic_homes.csv (rent from area and rooms), synthetic_fares.csv and the iris flowers. Real people's records are not used. Repeat the AI rules: any assisted step is logged in AI_USE_LOG.md after your own attempt, or the log says No AI assistance used.

Investigate, with answers:

- Which heading stops you using the test answers to choose a model? Answer: Evaluation. It states which rows are held back and which metric is used.
- Why is the home_id column not an input for homes? Answer: It is only a label. It carries no information about rent.

Modify: Write the leakage column you will exclude and why. Answer: For fares: tip_ugx, because it is known only after the trip.

Make: reference

An approved plan has a narrow question, a supplied dataset, a target present in the file, inputs known beforehand, a baseline fitted on training data, a metric and a held-back set, and two stated limits.

Build log entry to look for: PROJECT_PLAN.md with seven headings, one named leakage column, and the instructor's approval.

Misconceptions to listen for

- A highly correlated column is always a good input.
- The plan can change after I see the held-back result.
- The biggest model is the best project.

Questions to ask, with expected answers

- Why plan before coding? Expected: Choices made after seeing test results are no longer honest.
- What is the leakage column in fares? Expected: tip_ugx.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Tour the fares data: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, A plan needs seven headings: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Tour the fares data): Print the correlation of the fare with the trip_id column.
2. Modify, Block B (A plan needs seven headings): Write the leakage column you will exclude and why.
3. Make: Write PROJECT_PLAN.md using the seven headings. Name one leakage column and exclude it. Start from the topic on your progress map and choose the supplied dataset that fits it best. Real public data is used only after you and the instructor have read its licence. Ask the instructor to approve the scope before you build.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: Why plan before coding? Expected: Choices made after seeing test results are no longer honest.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML20: Build the project baseline

Private. Keep this file out of the student folder.

Objective. By the end: you can split data into training, validation and held-back parts, fit a baseline on training only, and draw one descriptive chart.

Question of the day. How much does a plain guess miss by, before any model helps?

> **WHERE THIS IS USED**
> A trader guessing next week's maize price might simply repeat this week's price. A model has to do better than that plain guess to be useful. What is the plain guess for your project?

Prerequisite: ML19 Project data and a safe plan.

The 180 minutes

Minutes	Activity
10	recall
40	split and baseline
15	break
35	chart
70	build on your own dataset
10	save and exit check

Where it overruns: Learners on other datasets need the same steps. Check the split sizes on each project before they go further.

Block A. Split and fit the baseline

Use the fares as the worked example, then repeat with your own dataset. Hold 20% back, take 25% of the rest for validation, and fit the mean guess on training only.

Ask for the prediction first: Predict the three sizes and whether the baseline will miss fares by more than UGX 3000 on average.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error
fares = pd.read_csv("synthetic_fares.csv")
X = fares[["distance_km", "rush_hour", "raining"]]
y = fares["fare_ugx"]
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.2, random_state=7)
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.25,
    random_state=7)
baseline = DummyRegressor(strategy="mean").fit(X_train, y_train)
val_baseline_mae = mean_absolute_error(y_val, baseline.predict(X_val))
print(len(X_train), len(X_val), len(X_test), round(val_baseline_mae))
```

Correct result: The cell prints 90, 30, 30 and 4361. The plain guess misses the validation fares by about UGX 4361 on average. Any model you build must beat that on the same validation trips. The baseline was fitted on the 90 training trips only. The 30 held-back trips stay untouched.

Investigate, with answers:

- Why is the baseline fitted on training only? Answer: Fitting it on all rows would let it see the validation and test answers.
- What would make the baseline number meaningless? Answer: Computing it on rows that were used to fit it.

Modify: Print the share of the 150 trips used for training. Answer: 0.6, which is 90 out of 150.

```
train_share = len(X_train) / len(fares)
```

Block B. One descriptive chart

Plot the training trips: distance on one axis, fare on the other. Label both with units.

Ask for the prediction first: Predict the shape of the dots: rising, falling or no pattern.

```
import seaborn as sns
import matplotlib.pyplot as plt
train = X_train.copy()
train["fare_ugx"] = y_train
sns.scatterplot(data=train, x="distance_km", y="fare_ugx")
plt.xlabel("Distance (km)")
plt.ylabel("Fare (UGX)")
plt.title("Fare against distance, training trips")
plt.show()
train_corr = train["distance_km"].corr(train["fare_ugx"])
```

Correct result: The dots rise from left to right with a lot of scatter. The correlation on the 90 training trips is about 0.76. Longer trips cost more, but two trips of equal length can differ by thousands of shillings. That scatter is why even the best line will still miss.

Investigate, with answers:

- What does the scatter around the rising pattern tell you? Answer: Distance is not the whole story, so the model will make errors.
- Why plot only the training trips? Answer: Looking at validation or test answers while choosing would leak information about them.

Modify: Colour the dots by rush_hour by adding hue="rush_hour" to the plot. Answer: Two colours appear, and they overlap a lot: rush hour changes the fare little.

```
sns.scatterplot(data=train, x="distance_km", y="fare_ugx", hue="rush_hour")
plt.show()
```

Make: reference

Check the source, the split sizes, a baseline fitted only on training rows, a metric in the target's unit, and a labelled chart. The learner explains every column used.

Build log entry to look for: project.ipynb with the split, the baseline error on validation, one labelled chart, a data dictionary and a cleaning log.

Misconceptions to listen for

- The baseline is a model I do not need to report.
- I can look at the test rows to understand the data.
- A chart should include all rows.

Questions to ask, with expected answers

- What is the baseline error for the fares example? Expected: About UGX 4361 on validation.
- Which rows may the chart use? Expected: Training rows.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Split and fit the baseline: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, One descriptive chart: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Split and fit the baseline): Print the share of the 150 trips used for training.
2. Modify, Block B (One descriptive chart): Colour the dots by rush_hour by adding hue="rush_hour" to the plot.
3. Make: Complete the three-way split, the baseline and one labelled chart for your own project dataset. Save the notebook, your data dictionary and the cleaning log. If no cleaning was needed, record the checks that found nothing.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What is the baseline error for the fares example? Expected: About UGX 4361 on validation.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML21: Compare, explain and critique

Private. Keep this file out of the student folder.

Objective. By the end: you can compare candidate models on the same validation rows, explain why one input is excluded, and check a coding assistant's advice with tests.

Question of the day. Does my model beat the baseline fairly, and can I defend each input?

> **WHERE THIS IS USED**
> When a model helps decide something about people, such as a loan, others ask why. The team must explain which inputs it used and defend them. Can you defend every input in your model?

Prerequisite: ML20 Build the project baseline.

The 180 minutes

Minutes	Activity
10	recall
45	build and compare
15	break
20	supervised coding-assistant critique
75	project work
15	save and exit check

Where it overruns: The 20-minute critique replaces part of the practice block. Close the assistant before returning to project work.

Block A. A fair comparison

Compare four options on the same 30 validation trips: the plain guess, distance only, three inputs, and three inputs plus tip. A small function keeps every comparison the same.

Ask for the prediction first: Predict which option has the lowest error, and whether you may use it.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
fares = pd.read_csv("synthetic_fares.csv")
y = fares["fare_ugx"]

def validation_mae(columns, model):
    X = fares[columns]
    X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.2, random_state=7)
    X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.25,
        random_state=7)
    model.fit(X_train, y_train)
    return mean_absolute_error(y_val, model.predict(X_val))
```

```
base = validation_mae(["distance_km"], DummyRegressor(strategy="mean"))
distance_only = validation_mae(["distance_km"], LinearRegression())
three = validation_mae(["distance_km", "rush_hour", "raining"], LinearRegression())
with_tip = validation_mae(["distance_km", "rush_hour", "raining", "tip_ugx"],
    LinearRegression())
print(round(base), round(distance_only), round(three), round(with_tip))
```

Correct result: The cell prints 4361, 2415, 2255 and 1317. The model with the tip has the lowest error, and it must not be used, because the tip is known only after the trip. Among the allowed models the three-input line wins: it misses by about UGX 2255, against 4361 for the plain guess. Every option used the same split, the same 30 validation trips and the same measure, which is what makes the comparison fair. The held-back set is untouched.

Investigate, with answers:

- Why is the lowest error not the winner here? Answer: The tip column leaks the answer. A model that cheats scores well and predicts nothing in real use.
- What stays the same in a fair comparison? Answer: The rows, the split, and the measure of error.

Modify: Try distance and raining only, and print the validation error. Answer: About UGX 2262, slightly worse than with rush hour too.

```
distance_rain = validation_mae(["distance_km", "raining"], LinearRegression())
```

Block B. Check an assistant's advice

A function should multiply a price by a quantity but gives the wrong answer. A tutor shows an assistant's hint, or a tutor-written one, and you test it. Use only this practice function; your project stays separate. You already know functions and assert from session 6.

Ask for the prediction first: Predict what total(2000, 3) gives with the plus sign, and what it should give.

```
def total(price, quantity):
    return price + quantity

buggy_result = total(2000, 3)
print(buggy_result)

def total(price, quantity):
    return price * quantity

assert total(2000, 3) == 6000
assert total(1000, 0) == 0
assert total(1000, 1) == 1000
```

Correct result: The first print shows 2003, because plus adds the numbers. The required answer is 6000, and multiplying fixes it. The three asserts pass silently. The tests support this small requirement and do not prove the function is right for every input. The tutor's hint says to inspect the arithmetic operator and to try total(2000, 3) and total(1000, 0). A claim that every bug in a total function is fixed by changing + to * goes too far: a different bug would need a different repair.

Investigate, with answers:

- What is useful in the hint and what is unjustified? Answer: Useful: inspect the operator and test with 0. Unjustified: the claim that every bug is fixed by changing + to *.
- Why test with a quantity of 0? Answer: A zero quantity must give a total of 0, which catches many mistakes.
- What goes in AI_USE_LOG.md? Answer: The date and tool, your own attempt, the question asked, the reply, what you accepted or rejected and why, and the tests you ran.

Modify: Add one more assert of your own and run the cell. Answer: For example assert total(500, 4) == 2000.

```
assert total(500, 4) == 2000
```

Make: reference

A model that does not beat the baseline is still a good result if the method is sound and you report it plainly. The partner can state the target, baseline and metric without asking.

Build log entry to look for: project.ipynb with the baseline error, one candidate model and its validation error, a written decision, and AI_USE_LOG.md.

Misconceptions to listen for

- The model with the best score is always the one to use.
- An assistant's advice is correct when it sounds confident.
- Passing three tests proves a function is right for all inputs.

Questions to ask, with expected answers

- Why is with_tip not a valid candidate? Expected: It uses information from after the trip.
- What does the AI log prove? Expected: That the learner checked and explained the advice.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, A fair comparison: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, Check an assistant's advice: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (A fair comparison): Try distance and raining only, and print the validation error.
2. Modify, Block B (Check an assistant's advice): Add one more assert of your own and run the cell.
3. Make: Add model code, the validation error and a short decision to project.ipynb. Ask a partner to name your target and baseline from the notebook alone. Record any assistance in AI_USE_LOG.md or write No AI assistance used.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: Why is with_tip not a valid candidate? Expected: It uses information from after the trip.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML22: The final run: your project, ready to share

Private. Keep this file out of the student folder.

Objective. By the end: you can run the held-back check once, report it separately from validation, and share a project that reruns from a restarted kernel.

Question of the day. How well does my frozen model do on rows nobody has used?

> **WHERE THIS IS USED**
> Before a model goes live, a team freezes it, tests it once on untouched data and packs the work so someone else can rerun it. Today you do the same. Could a stranger rerun your project tomorrow?

Prerequisite: ML21 Compare, explain and critique.

The 180 minutes

Minutes	Activity
10	recall
40	held-back run
15	break
25	handover checklist
80	assemble, rerun and report
10	save and exit check

Where it overruns: Learners with different datasets need the same held-back discipline. Check each learner's order of steps.

Block A. Run the held-back check once

The fares model with three inputs was chosen on validation. Now run the 30 held-back trips once.

Ask for the prediction first: Predict whether the held-back error will be above or below the validation error of 2255.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error
fares = pd.read_csv("synthetic_fares.csv")
X = fares[["distance_km", "rush_hour", "raining"]]
y = fares["fare_ugx"]
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.2, random_state=7)
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.25,
    random_state=7)
from sklearn.linear_model import LinearRegression
baseline = DummyRegressor(strategy="mean").fit(X_train, y_train)
model = LinearRegression().fit(X_train, y_train)
final_mae = mean_absolute_error(y_test, model.predict(X_test))
```

```
final_baseline_mae = mean_absolute_error(y_test, baseline.predict(X_test))
print(round(final_mae), round(final_baseline_mae))
```

Correct result: The cell prints 2936 and 3617. On the 30 held-back trips the model misses by about UGX 2936 and the plain guess by about UGX 3617. The model is still better, but its lead is much smaller than the validation lead of 2255 against 4361. This is normal: the held-back set has only 30 trips, and the validation lead may have been lucky. Report the final number as it is, next to the validation number, and do not change the model to improve it.

Investigate, with answers:

- Why report the final score separately from validation? Answer: Validation was used to choose the model, so only the held-back set gives an honest score.
- Can you change the model if the final score is disappointing? Answer: No. Changing it after seeing the result would use those trips twice, and a new held-back set would be needed.

Modify: Calculate how much smaller the model's error is than the baseline's on the held-back trips, as a percentage. Answer: About 19% smaller.

```
improvement = (final_baseline_mae - final_mae) / final_baseline_mae * 100
```

Block B. The handover checklist

Another person should be able to rerun your folder. Work through the list with a partner.

Ask for the prediction first: Write the three things you think a stranger needs first to rerun your project.

Correct result: The checklist: restart the kernel and run all cells from the top; the data file sits beside the notebook; the data origin and the fact that it is synthetic are stated; the held-back result is reported separately from validation; limits are written next to the result; project.ipynb, the data dictionary and report.pdf or report.docx are saved together; every file opens when reopened. A model on invented teaching data is a demonstration, and nobody should use it to make decisions about people.

Investigate, with answers:

- Why restart the kernel before the final run? Answer: Hidden steps are exposed when the notebook runs from a clean start.
- What does the report say about the model's limits? Answer: The data is invented, the held-back set is small, and the model has not been tried on real trips.

Modify: Swap notebooks with a partner. Follow their instructions without asking questions and note where you stall. Answer: Every stall is a missing instruction. Add it.

Make: reference

All files open and agree with each other. The held-back result is clearly separate from validation. Nothing relies on a variable that exists only in an earlier session.

Build log entry to look for: A complete local project folder and a clean rerun from a restarted kernel, with the held-back result reported once.

Misconceptions to listen for

- A disappointing final score means I should retune.
- The notebook works if it ran once.
- The report only needs the best number.

Questions to ask, with expected answers

- What does the held-back result tell you that validation does not? Expected: How the frozen model does on rows no choice ever used.
- What is the first fix when a notebook breaks on rerun? Expected: Restart and run all, and find the first cell that fails.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Run the held-back check once: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, The handover checklist: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Run the held-back check once): Calculate how much smaller the model's error is than the baseline's on the held-back trips, as a percentage.
2. Modify, Block B (The handover checklist): Swap notebooks with a partner. Follow their instructions without asking questions and note where you stall.
3. Make: Use the project checklist. Reopen all your files. Save project.ipynb, your data dictionary and report.pdf or report.docx together in your project folder, and share the folder with the instructor by the route in START_HERE.md.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What does the held-back result tell you that validation does not? Expected: How the frozen model does on rows no choice ever used.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML23: Explain the project to a parent

Private. Keep this file out of the student folder.

Objective. By the end: you can explain your project in five minutes with five slides, one demonstration, one chart and one limit, without jargon.

Question of the day. Can a parent say what my model predicts, how I tested it and why it is limited?

> **WHERE THIS IS USED**
> Part of every data job is explaining results to people who do not code, such as managers, farmers or clinic staff. Can you explain your model to a parent in words a ten-year-old would follow?

Prerequisite: ML22 The final run: your project, ready to share.

The 180 minutes

Minutes	Activity
10	recall
40	five slides
15	break
50	rehearsals with a timer
20	optional modern-AI orientation or second rehearsal
30	individual feedback
15	save backups and exit check

Where it overruns: The optional orientation is 20 minutes after the rehearsal. Skip it if any learner still needs rehearsal support.

Block A. Five slides, five minutes

Build five slides: my question; what data I used; what I built; my result against the baseline; the limit and the next step. Say what a prediction, a feature and a target are in plain words.

Ask for the prediction first: Write on a card, in one sentence each, what a prediction, a feature and a target mean for your project.

Correct result: A listener should be able to say what was predicted, how it was tested and why the result is limited. One chart and one worked prediction say more than a list of library names. Show only approved, non-personal data. Prepare a screenshot backup and disclose any assistance. Publishing online is optional and needs agreement.

Investigate, with answers:

- What makes a slide too crowded? Answer: More than one idea, or code the audience cannot read from the back.

- What belongs on the limit slide? Answer: That the data is invented, the held-back set is small, and the model is a demonstration.

Modify: Time yourself giving the five slides without looking at notes. Answer: Five minutes is enough for one question, one demonstration and one chart. Cut anything that takes more.

Block B. Rehearse and receive feedback

Give the talk to a partner and answer one question. Your partner says back what the model predicts and how you tested it.

Ask for the prediction first: Predict the question a parent is most likely to ask.

Correct result: Common parent questions: where did this number come from, what did you leave out, and what cannot this model tell us. Answer by pointing to the chart or the table. If the partner cannot repeat your target and baseline, replace explanation with a worked example.

Investigate, with answers:

- If the explanation is mostly code, what should replace it? Answer: The question, a labelled chart and a worked prediction.
- Is it fine to say I do not know? Answer: Yes, and say what you would check next.

Modify: Say your project's limit in one sentence a younger student would follow. Answer: For example: The fares are invented, so this shows how a model works and not what real riders pay.

Make: reference

A listener can say what was predicted, how it was tested and why the result is limited. The backup opens without the laptop.

Build log entry to look for: Five slides saved as showcase.pptx, a PDF or screenshot backup, and a timed rehearsal with feedback.

Misconceptions to listen for

- More detail makes a better talk.
- The code is the most important slide.
- A limitation weakens my project.

Questions to ask, with expected answers

- What do parents need to hear first? Expected: What the model predicts, in everyday words.
- When can the modern-AI orientation be used? Expected: After the rehearsal, if every learner is ready. It replaces the optional second rehearsal and adds no assignment.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

Live clock, 90 minutes

Minutes	Activity
5	Recall: three questions on paper, then say the answers aloud
35	Block A, Five slides, five minutes: predict, run, investigate. The Modify step moves to the build task.
5	Break away from the screen
35	Block B, Rehearse and receive feedback: predict, run, investigate. The Modify step moves to the build task.
10	Close: say the exit sentence aloud, then the build task is set

After-class build task, about an hour

1. Modify, Block A (Five slides, five minutes): Time yourself giving the five slides without looking at notes.
2. Modify, Block B (Rehearse and receive feedback): Say your project's limit in one sentence a younger student would follow.
3. Make: Practise defining prediction, feature and target without jargon. Save a screenshot backup of your demonstration and disclose any AI assistance.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

Next class

In the next class recall, ask: What do parents need to hear first? Expected: What the model predicts, in everyday words.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

ML24: Parent showcase and next steps

Private. Keep this file out of the student folder.

Objective. By the end: you can present your project, explain one decision, try one small change live with no AI assistant open, and name your next learning goal.

Question of the day. Can I show my project and explain how I built it?

> **WHERE THIS IS USED**
> Start-ups and tech hubs hold demo days where builders show what they made in a few minutes and answer questions. Today is your demo day. What do you want your family to remember?

Prerequisite: ML23 Explain the project to a parent.

The 180 minutes

Minutes	Activity
30	final setup and screenshot backup check
10	welcome
40	presentations and questions
15	break
25	for one small live change by each learner
20	shared feedback and certificates
40	project folder, reflection and next-step plan

Where it overruns: For five learners the presentations fit in 40 minutes. A group of 15 needs a separate event slot.

Block A. Present and explain

Each learner has five minutes to present and three minutes for questions. Explain the input, the output and one failure case.

Ask for the prediction first: Write the one decision in your project you will explain if asked.

Correct result: Every learner follows the same friendly order. You show your own work, explain one decision, and answer a question from the audience about your code. Close assistant windows and turn off generative suggestions while you present. If the technology fails, use the screenshot backup and explain the steps; a short live rerun can be arranged later.

Investigate, with answers:

- What is a failure case? Answer: An input where the model does badly, with a reason.
- What if the laptop fails? Answer: Show the backup and explain the workflow. Rerun later if evidence is missing.

Modify: Say one thing your model gets wrong and why. Answer: For example: it misses fares for very long trips, because it learned from short ones.

Block B. A small change, live

The instructor suggests one small change to your code, such as a different split size or one more input. Make it and say what you expect.

Ask for the prediction first: Before the change, predict whether your error will go up or down.

Correct result: The change is small on purpose. It is a chance to show how your notebook works. The programme certificate goes to learners who attended or made up most classes, kept their build log and showed a finished project. The camp has no tests and no scores. The certificate is issued by the programme provider and is not a university qualification or a certificate from another course provider.

Investigate, with answers:

- What does the certificate show? Answer: That you finished the track: you came to most classes, kept your build log and showed a project.
- What is a good next learning goal? Answer: One specific thing, such as a second model on the same data, or a chart you did not finish.

Modify: Write your next learning goal on one line. Answer: A specific goal you can start next week.

Make: reference

The instructor gives one strength and one next step, kindly and privately, and notes that the learner finished the programme.

Build log entry to look for: A showcase talk, one small live change, one strength and one next step, and a certificate decision based on attendance, build log and finished project.

Misconceptions to listen for

- The certificate is a university qualification.
- I need AI to change my code.
- A mistake while showing my project means I did badly.

Questions to ask, with expected answers

- What do the instructor and the audience look for in the showcase? Expected: That you can explain your own work, through one decision and one small change.
- What if I missed some classes? Expected: Make them up with the instructor. The certificate needs most classes attended or made up.

Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

The showcase is best held in person. Use this clock only when a family cannot come. No recording without written consent.

Live clock, 90 minutes

Minutes	Activity
15	Tech check, screen share test and the backup PDF ready on each laptop
5	Welcome for families on the video call
45	Presentations and questions: about nine minutes for each learner, five minutes of talk and the rest for questions
15	Feedback for every learner: one strength and one next step, then the next learning goal
10	Close: thanks, the project folder is sent by the route the organiser sets

After-class build task, about an hour

No build task. Send the project folder by the route the organiser sets, and keep a backup copy.

Next class

There is no next class. Use the first minutes of any follow-up call to ask what each learner will learn next.

Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.